The Impact of AI-generated Code on the Future of Junior Developers

Bachelor's thesis

Computer Applications

Fall 2024

Carlos Pantin

HAMK
Häme University
of Applied Sciences

Degree Programme in Computer Applications
Author Carlos Pantin
Subject The Impact of AI-Generated Code on the Future of Junior Developers
Supervisors Elina Kulmala

Abstract
Year 2024

This thesis explores how artificial intelligence impacts junior developers as well as the software development industry. Artificial intelligence has been increasing in popularity recently. With the introduction of generative AI, plenty of new tools and technology are available for software developers. This thesis is composed of three main pillars. First, a theoretical research aimed at providing a better understanding of how this technology will impact developers and how it can be used to empower them. The second pillar consists of an experiment designed to test AI's effectiveness in solving certain difficult problems. Lastly, a survey to gather more information on students and their use patterns with this technology.

Results of the thesis demonstrate that artificial intelligence tools such as ChatGPT are efficient at solving certain programming programs, however, this heavily depends on the complexity of the problem among other factors. Artificial intelligence struggles with high-complexity problems and those types of problems are the standard in the workplace. However, this tool demonstrates that it can be of big aid to developers. This tool can help developers complete tasks up to twice as fast. However, the developer needs to constantly collaborate with the tool and revise its output to ensure correctness and maintain good code quality. HAMK's computer applications students expressed that they are starting to incorporate this tool within their programming tasks as well as frequent use of it.

While artificial intelligence is changing the industry and new advancements are introduced as time goes by, this technology is far from perfect. Software development is more than simply writing a few lines of code. To be successful in this industry, skills such as creativity, problem-solving skills, and critical thinking are needed. Moreover, software developers constantly work together on a team as well as know further context needed for a specific project unknown to artificial intelligence.

The outcome of the thesis explains how artificial intelligence is not a full-on replacement of junior developers, but more of a collaboration between the two, a syngery. Artificial intelligence can make developers more effective at what they do, however, without a developer's oversight or collaboration, the tool lacks plenty of characteristics essential to tackle tasks that a developer works on.

Keywords Artificial Intelligence, Junior Developers, Software development, AI-generated code.
Pages 67 pages and appendices 5 pages

Content

Figures, tables, and equations

Appendices

1 Introduction

Artificial Intelligence (AI) is a field of knowledge that seeks to create computer systems, robots, or products that can imitate human thinking. Within the scientific community, AI is an area of intense study that is comprised of multiple branches. Over the years, machine learning and optimization have been intensely researched, this lead to the rise of innovative and complex topics like transfer optimization, swarm robotics, and real-time drift detection and adaptation. Additionally, the creation of new approaches such as deep learning and explainable AI has reinforced AI's status as a key concept of computer science. (Osaba, Villar, Lobo, & Laña, 2021, p. 1)

According to Gwerzman (2023), the nature of the software development industry goes through constant evolution, especially with the rise of Artificial Intelligence (AI). This technological advancement has accelerated the pace of change within the industry, particularly with the integration of generative AI and its ability to generate code and other automation tools. Due to this, the traditional methods of software development are being revolutionized, impacting the roles and responsibilities of junior software developers.

As Artificial Intelligence (AI) increasingly handles routine tasks, junior developers may encounter challenges such as job displacement and limited opportunities to gain experience and progress in their careers. Nevertheless, this scenario also offers an opportunity for them to pivot, adapt, and explore innovative avenues to maintain their relevance and value in the evolving market. (Gwerzman, 2023).

The goal of this thesis is to explore the impact of AI on the future of junior developers and the software development industry as a whole as well as the effectiveness of generative AI on tackling coding tasks. Specifically, the thesis will examine the potential benefits and drawbacks of this technology and how it may affect the career paths and opportunities of junior developers. The research questions for the thesis are as follows:

- How effective is generative AI in generating code, ranging from solving easy to complex programming tasks?
- In what ways can AI-generated code be leveraged as an empowering tool for junior developers?
- How do students use, integrate, and perceive AI generative tools within the software development industry?

2 Artificial Intelligence

In this chapter, the thesis examines Artificial Intelligence (AI) and its applications. To begin, the foundational principles of AI and its various branches, methodologies, history, and recent advancements are examined. Through a comprehensive theoretical analysis, this chapter aims to provide a comprehensive understanding of AI This chapter will help set the ground for further discussion of chatbots and artificial intelligence-generated code within the next chapter of this thesis.

2.1 History of Artificial Intelligence

The origins of Artificial Intelligence (AI) can be traced back to the 1940s, with a notable milestone being the publication of Isaac Asimov's short story "Runaround" in 1942. Asimov's story explored the Three Laws of Robotics, which state that a robot must prioritize the safety of humans, follow human orders within reason, and protect its existence without endangering humans. Asimov's work has been a source of inspiration for generations of researchers and innovators in the fields of robotics, AI, and computer science. (Haenlein & Kaplan, 2019, p. 6)

Around the same time, English mathematician Alan Turing was tasked with much more practical problems. He designed a code-breaking machine named The Bombe for the British government to decode the Enigma code used by the German army during the Second World War, a task previously impossible to even the best mathematicians at the time. The Bombe is considered the first ever working electro-mechanical computer, and this made Turing wonder about the intelligence of the machines. Then, in 1950, Turing published an influential article titled "Computing Machinery and Intelligence." In the article, he proposed a method for creating intelligent machines and outlined how to test their intelligence. This method later became known as the Turing Test, which is still considered a benchmark for identifying the intelligence of an artificial system. The way the Turing test works is a following, if a person interacts with both another human and a machine and the person cannot tell which is which, then the machine is considered intelligent. (Haenlein & Kaplan, 2019, p. 6-7)

However, the word Artificial intelligence was officially claimed about six years later when, in 1957, computer scientists John McCarthy and Marvin Minsky hosted the Dartmouth Summer Research Project on Artificial Intelligence (D.S.R.P.A.I.). This event reunited those who later became the founding fathers of AI The goal of D.S.R.P.A.I. was to bring together researchers

from diverse fields to create a new research area focused on developing machines capable of simulating human intelligence. (Haenlein & Kaplan, 2019, p. 6-7)

After the Dartmouth Conference, it was almost twenty years after AI research achieved significant progress. One of the early examples was the E.L.I.Z.A. computer program, developed by Joseph Weizenbaum at M.I.T. between 1964 and 1966. E.L.I.Z.A. was a natural language processing tool that could simulate human conversation and was one of the first programs attempting to pass the Turing Test. (Haenlein & Kaplan, 2019, p. 7)

During the early days of AI, another success story emerged. A program called General Problem Solver was developed by Nobel Prize winner Herbert Simon, along with scientists from RAND Corporation, Cliff Shaw and Allen Newell. This program was capable of automatically solving simple problems like the Towers of Hanoi. It was a remarkable success story in the field of AI and thanks to these successes, significant funding was given to the research of this emerging technology, leading to more and more projects. (Haenlein & Kaplan, 2019, p.7)

In 1973, the U.S. Congress began to heavily criticize the large expenditures on research in the field of Artificial Intelligence (AI). That same year, British mathematician James Lighthill released a report that had been commissioned by the British Science Research Council. In the report, Lighthill raised doubts about the optimistic predictions made by AI researchers that a machine can achieve with general human intelligence could be developed. The British government terminated further support for the research of Artificial Intelligence (AI) in all but three universities. The United States government soon followed the British steps. The Japanese government heavily funded AI research in the 1980s. In response, the US DARPA also increased funding, but no significant advances were made in the following years. (Haenlein & Kaplan, 2019, p. 7-8)

One reason for the initial slow progress in the field of AI can be attributed to the specific approach that early systems like E.L.I.Z.A. and the General Problem Solver attempted to imitate human intelligence. The problem was that these approaches were Expert Systems. Expert systems are collections of rules that assume that human intelligence can be reconstructed in a top-down approach as a series of if-then statements. These expert systems can outstandingly perform in some areas that behave like this. An example of this is the I.B.M.'s Deep Blue chess program. In 1997, Deep Blue was able to beat the world chess champion Gary Kasparov. (Haenlein & Kaplan, 2019, p. 8)

As mentioned above, expert systems only perform well in areas that have that particular behavior. For example, an expert system cannot be easily trained to recognize faces or distinguish a picture showing a muffin and one picture of a Chihuahua. For these types of tasks, a system must be able to interpret external data correctly, learn from the data, and use those learnings. (Haenlein & Kaplan, 2019, p. 8)

Statistical methods have been discussed since the 1940s to achieve true AI Around that decade, Canadian psychologist Donald Hebb developed a theory of learning known as Hebbian Learning. This theory replicates the process of neurons in the human brain. This theory led to the research of Artificial Neural Networks. However, work for this came to an impasse in 1969 when Marvin Minsky and Seymour Papert proved that computers needed more processing power to handle work for such artificial neural networks. (Haenlein & Kaplan, 2019, p. 8)

However, artificial neural networks made a comeback in the form of Deep Learning in 2015. During this year, a program developed by Google called AlphaGo was able to defeat a world champion in the board game Go. AlphaGo accomplished this feat by using a specific type of artificial neural network called Deep Learning. Nowadays, these networks and deep learning form the basis of most applications under the AI label. (Haenlein & Kaplan, 2019, p. 8)

2.2 Definition of Artificial Intelligence

Answering the question about artificial intelligence (AI) proves challenging due to two main factors. Firstly, there needs to be more consensus on what intelligence truly means. Secondly, the correlation between machine intelligence and human intelligence remains uncertain. Various definitions of AI exist, each emphasizing the development of computer programs or machines capable of intelligent behavior similar to humans. John McCarthy, a pioneer in the field, described AI as the endeavor to make machines exhibit behavior that would be deemed intelligent if performed by a human. (Kaplan, 2016)

This encompasses the development of systems equipped with cognitive abilities, such as reasoning, understanding context, generalization, and learning from previous experiences. Computers demonstrate remarkable proficiency in executing complex tasks. However, despite advancements in processing power and memory capacity, existing programs still fall short of replicating the full strength of human cognitive flexibility across different domains or in tasks requiring extensive everyday knowledge. The pursuit of such comprehensive

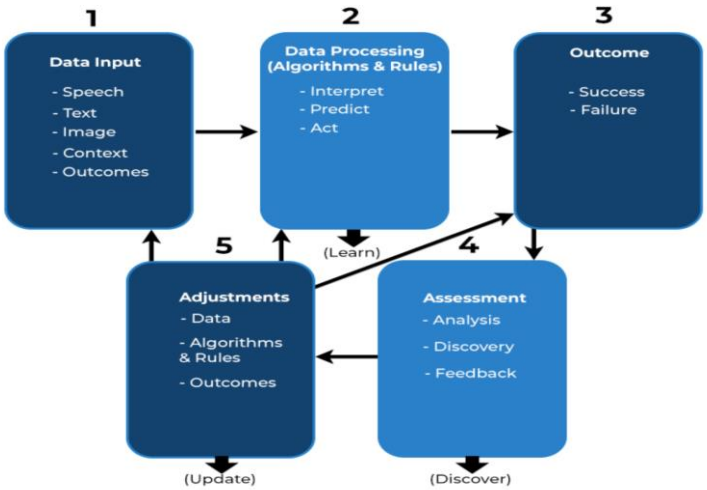
human-like intelligence is referred to as artificial general intelligence (A.G.I.) or strong AI (Britannica, T. Editors of Encyclopedia, 2024)

Nowadays, artificial intelligence (AI) has become a technology very used throughout our day-to-day life as well as in industrial operations alike. Its integration is clear in mobile applications, enriching user interactions and guiding recommendations on online shopping platforms. Moreover, AI's reach extends to a wide range of industries, where it powers predictive analytics, decision support systems, and artificial vision technologies. These advancements facilitate various functions, such as medical diagnostics, automated inspections, robotics, and advanced manufacturing, among others. However, despite the widespread adoption of AI, only the surface of a technological revolution has been scratched. It is widely recognized that companies proficient in AI technologies will gain a competitive edge in an increasingly interconnected and automated world, positioning themselves as market leaders. (Osaba, Villar, Lobo, & Laña, 2021, p. 1)

2.2.1 Functionalities of AI

To begin with, an AI system takes in data in various forms like speech, text, or images. Then, it processes this data using different rules and algorithms, making sense of it, making predictions, and acting accordingly. After processing the data, the system produces a result indicating whether it succeeded or failed based on the input data. This result is then evaluated through analysis and feedback. Finally, the system uses this feedback to tweak its input data, rules, and algorithms until it achieves the desired outcome. This iterative process continues until the desired result is reached. (Kanade, 2022). Figure 1, as shown below, gives a visual view of the explanation of this technology.

Figure 1. Basic explanation of artificial intelligence. (Kanade, 2022)



AI utilizes key tools like machine learning and deep learning to execute tasks. It is important to note that these tasks are executed while trying to mirror that of the human mind.

Machine learning represents a particular application of AI, enabling computer systems or programs to autonomously acquire knowledge and produce outcomes based on previous experiences. ML algorithms operate by analyzing input data and employing diverse statistical methodologies, facilitating AI in uncovering patterns within the data and thereby enhancing task outcomes. Deep learning is a type of machine learning that uses artificial neural networks to analyze data and reach conclusions. It is a specialized form of machine learning that allows AI systems to improve their capabilities through data processing. Deep learning models artificial neural networks based on the neural networks in the human brain. By identifying connections within the data, deep learning generates inferences or outcomes through positive and negative reinforcement. (Maheshwari, 2023)

Neural networks work similarly to networks of neurons found in the human brain, enabling AI to analyze vast datasets and delve deeply into drawing references, making connections, and weighing input for efficient results. Other than machine learning (ML) and deep learning (DL), AI systems require robotics, cognitive computing abilities, natural language processing, and computer vision. These components enable computer models to mimic the cognitive processes of the human brain while executing intricate tasks. (Maheshwari, 2023)

ML algorithms encompass both descriptive and predictive techniques. Descriptive methods aim to characterize the data by summarizing or categorizing it. Alternatively, predictive analysis focuses on identifying trends, behaviors, or insights that could aid in predicting

future outcomes. ML algorithms are further classified based on the learning style used for their model, typically categorized as supervised, unsupervised, semi-supervised, and reinforcement learning. (Osaba, Villar, Lobo, & Laña, 2021, p. 2)

Supervised Learning is a particular type of machine learning that employs labeled input data to train the learning algorithm. In the training phase, an inferred function or model is generated to minimize an error function or maximize precision. These systems are created to make precise predictions for unseen examples. Such learning primarily tackles classification and regression issues. (Osaba, Villar, Lobo, & Laña, 2021, p. 2)

Unsupervised Learning does not provide any labeled input vector. The main goal of this type of learning is to identify the structure behind the patterns without any guidance or reward signal. These models examine and deduce peculiarities or common traits in the instances to discover similarities and associations among the samples. Examples of such problems are clustering and latent variable models. (Osaba, Villar, Lobo, & Laña, 2021, p. 2)

Semi-supervised learning is a method in which both labeled and unlabeled data are fed into the algorithm. This method falls between the categories of supervised and unsupervised learning. Acquiring labeled data can be expensive and often requires human skills, while unlabeled data can be of great practical value. The goal of semi-supervised learning systems can be either transductive, where analogies are searched to derive the labels of the unlabeled data, or inductive, where the mapping from initially labeled vectors to their corresponding categories is inferred. (Osaba, Villar, Lobo, & Laña, 2021, p. 2)

Reinforcement Learning falls under the category of machine learning, where the system interacts with the environment by taking action and receiving positive or negative feedback in response. This feedback prompts the learning process, which aims to minimize the punishment or maximize the reward. This type of learning is commonly used in robotics, where the algorithms identify relevant peripheral events and translate the feedback into a learning process. These algorithms are particularly useful in realistic environments that involve identifying relevant sensory inputs. (Osaba, Villar, Lobo, & Laña, 2021, p. 2)

2.2.2 Machine Learning

Machine learning (ML) is a system's capability to independently learn, combine, and enhance knowledge from extensive datasets. It allows the system to discover new patterns and insights on its own, without the need for direct programming. Simply put, ML algorithms can

be applied to achieve the following: gain deeper insights into the event that produced the data, encapsulate this understanding in a model, forecast future values based on the model, and proactively identify any unusual behavior and take actions based on that. (Sen, 2021, preface)

Machine learning (ML) is a constantly evolving field. With the development of more intelligent algorithms and improvements in hardware and storage systems, ML can now perform many tasks more efficiently and accurately than ever before. One specialized subset of ML is deep learning (DL). This approach uses more sophisticated architectures, algorithms, and models to address complex problems and predict future outcomes of complex events. (Sen, 2021, preface)

In recent times, according to Jaydip Sen (2021), there have been notable developments in machine learning algorithms, especially in fields like reinforcement learning, natural language processing, computer and robotic vision, image analysis, as well as speech and emotion recognition and comprehension. These developments have led to various machine learning applications that are emerging or evolving in several business domains, including medicine and healthcare, finance and investment, sales and marketing, operations and supply chain, human resources, media, and entertainment, among others. (Sen, 2021, preface)

2.2.3 Types of Artificial Intelligence

According to Betz (2024), depending on the ability of artificial intelligence to learn and apply such learning, artificial intelligence can be classified into three main categories. These categories include Narrow AI (ANI), General AI (A.G.I.), and Super AI (A.S.I.).

Applications that are designed to perform narrowly defined tasks or carry out specific tasks are known as Narrow AI (ANI) or weak AI Narrow AI cannot acquire skills beyond those made in its design, ANI technologies are also developed to excel in one cognitive capability only. This type of AI typically uses machine learning and neural network algorithms to accomplish these predefined tasks. (Betz, 2024)

Ray Kurzweil (2005) used the term narrow AI to refer to the creation of systems that carry out specific intelligent tasks in specific contexts. When the context or behavior specification of a narrow AI system is changed even slightly, it requires human reprogramming or reconfiguration to maintain its level of intelligence. Quite the opposite occurs with generally intelligent systems like humans. Humans possess a diverse range of abilities that allow them

to adapt to changes in their goals or environment. They are capable of transfer learning, which involves applying knowledge from one context to another, as explained by (Taylor, Kuhlmann, and Stone, 2008). (Goertzel, 2013, p. 1)

According to Goertzel (2013), the term Artificial General Intelligence emerged to oppose that of narrow artificial intelligence to refer to systems with this sort of broad generalization capability.

Artificial General Intelligence (A.G.I.), also known as General AI or Strong AI, refers to a type of AI that can learn, reason, and perform various tasks similar to humans. The goal of developing artificial general intelligence is to create machines that can perform multiple tasks and support humans in their daily lives. Although it is still a work in progress, artificial general intelligence can be built using technologies like supercomputers, quantum hardware, and generative AI models such as ChatGPT. (Betz, 2024)

Now, artificial superintelligence (A.S.I.) is, for the time being, just a theory. This super AI could have intellectual capabilities superior to that of human intelligence. It is important to note that there are some technological building blocks for A.S.I., but A.S.I. itself is still hypothetical. Also important to note that the current level of AI is referred to as Artificial Narrow Intelligence (ANI). (Mucci & Stryker, 2023)

So far, there has yet to be a saying amongst intellectuals regarding the possibility of creating artificial superintelligence. Human intelligence is the result of complex evolutionary processes, and by definition, it may not be the most efficient or universal form of intelligence. Also, the human brain has yet to be completely understood, and that makes it challenging to replicate through software and hardware. (Mucci & Stryker 2023)

2.3 Ethics of Artificial Intelligence

According to UNESCO (2024), "Getting AI governance right is one of the most consequential challenges of our time, calling for mutual learning based on the lessons and good practices emerging from the different jurisdictions around the world."

Before starting, the grounds of ethics need to be explained.

Ethics is the foundation of well established principles of right and wrong that guide what people should do. These principles often relate to fundamental aspects such as individual

rights, responsibilities, the well-being of society, fairness, or specific virtues like honesty and integrity. Ethics serves as a framework for determining the proper course of action in various situations, helping people and communities navigate complex moral decisions. (Velasquez, Andre, Shanks, S, & Meyer, 2010)

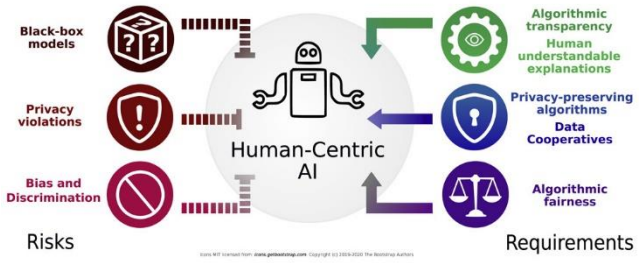
One of the most significant subjects in the 21st century is artificial intelligence, and with it comes ethical challenges. Artificial intelligence is known to offer benefits, such as reducing human error or using robots in risk-type situations. However, many ethical concerns are evolving in AI, including algorithmic bias, the digital divide, and severe health and safety issues. (Doris, Rowena, & Carsten 2023, p. 1)

Artificial intelligence is a general-purpose technology, and its applications are limited by our imagination and capabilities. As such, these technologies have unintended consequences, as demonstrated by past revolutionizing technology, whose consequences have been felt decades or even centuries into the future, and all such technologies have the potential for abuse and exploitation. (Wooldridge, 2020, p. 263)

A couple of examples of past technology being created and innovated and the potential consequences not foreseen can be our prehistoric ancestors who first discovered fire. Indeed, they did not anticipate the climactic changes in fossil fuels this would bring, the inventor of the electric generator in 1831. Michael Faraday probably did not anticipate the use of this invention in the electric chair. The inventor of the automobile, Karl Benz, in 1886, indeed did not foresee all of the automobility-related deaths his invention caused a century into the future. (Wooldridge, 2020, p. 263)

As these examples mentioned, AI will surely have adverse consequences, and these consequences will be felt at a global scale and will be abused.

Figure 2. Risks and requirements on ethics with artificial intelligence. (Lepri, Oliver, & Pentland, 2021)



According to the Assistant Director-General for Social and Human Sciences of UNESCO, Gabriela Ramos (2024), the ethical compass is more relevant to artificial intelligence than any other field. These technologies are reshaping the way work is done, interactions take place, and lives are lived, leading to rapid changes in the world. AI brings many benefits in plenty of areas, but without adequate ethical guardrails, there is a considerable risk of producing real-world biases and discrimination, fueling divisions and a threat to human rights. Figure 2 provides an overview of the above mentioned.

According to UNESCO (2024), there are, at the moment, four core values that lay the foundations for artificial intelligence. These four values include.

- Human rights and human dignity
- Living in peaceful
- Ensuring diversity and inclusiveness
- Environment and ecosystem flourishing

3 Software Development Industry

Software development has been the center of innovation and progress in the world of technology. This industry involves a systematic process of designing, coding, testing, and maintaining software applications and systems alike. From simple mobile applications to complex enterprise systems and applications, software development plays a vital role in the digital world.

Designing, creating, maintaining, testing, and building software applications is known as software development. Software development goes beyond these mentioned points; software development involves the application of principles in other fields, such as engineering, computer science, and mathematical analysis. In simpler words, software development's goal is to create efficient and user-friendly software. (Simplilearn, 2023)

3.1.1 Software Development Methodologies

Project completion involves the use of a system specific to each project. For instance, a software project has various development methods that depend on the nature and size of the project. Choosing the proper method is an important decision that requires time and consideration of the project's time and budget limitations. While numerous software development methodologies exist, it is challenging for project managers to know all of them. To solve this problem, in the 1960s, significant computers were used to create flowcharts that defined the stages of the method. However, technology is constantly evolving, and the methodologies that are used today may be irrelevant tomorrow. All of this is happening while the technology age rapidly changes year after year. (Saeed, Naqvi, & Humayun, 2019, p. 445)

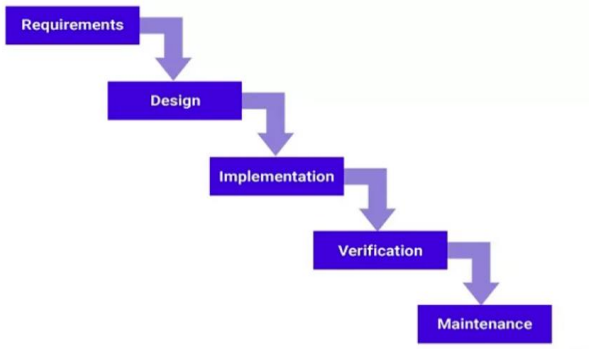
Below is an overview of the most used software development methodologies and why different methodologies exist.

Agile development methodology can be used by teams to reduce risks like bugs, cost overruns, and evolving requirements while adding new features. All agile approaches involve developing software in iterations, with each iteration delivering small increments of new functionality. (McGuire, 2024)

Figure 3. Agile methodology workflow. (Laoyan, 2024)

The waterfall methodology is considered by many to be the most traditional software development method. This method is a linear model that consists of sequential phases that each focus on different goals. Before the next step can be started, the previous step needs to be 100% completed. Usually, there is no process for going back and modifying the project or direction of it. (McGuire, 2024)

Figure 4. Waterfall methodology workflow. (McGuire, 2024)



DevOps is not just a development methodology. DevOps is also considered to be a set of practices that supports an organization's culture. DevOps deployment involves improving collaboration between departments responsible for different segments of the development life cycle, such as development, quality assurance, and operations. (McGuire, 2024)

3.1.2 Current State of Software Development

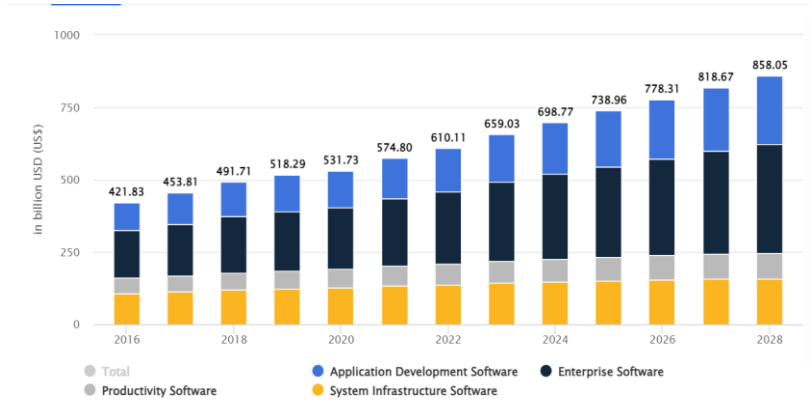
The software development industry is constantly evolving. This industry is being affected by new technologies and several other external factors, as well as new societal demands. AI plays a critical role in these changes, and this technology can shift the software development industry.

Global factors, including COVID-19, have made a considerable impact on the software development industry. Software providers were forced to cut investments, project delays, and staffing reductions. All of this resulted in a negative growth rate for the industry. (Sakovich, 2024)

Despite this negative shift. In 2021, the industry shifted into positive numbers, and the I.T. sector returned to a growth trajectory. Both small and large businesses have recognized the importance of transitioning to a primarily digital landscape. (Sakovich, 2024)

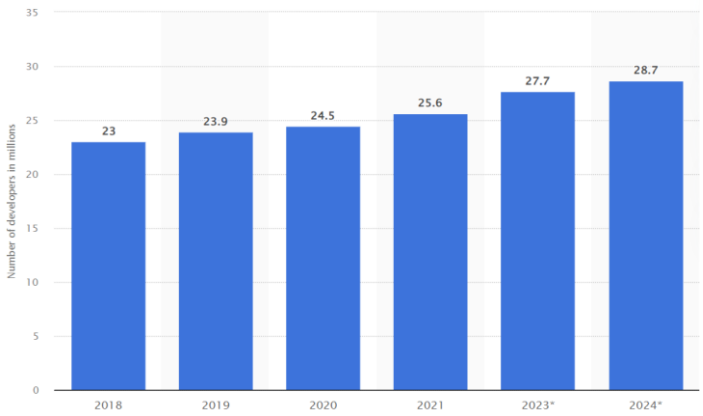
According to a survey conducted by Accelerated Strategies Group, 63.3% of businesses prioritized accelerating their digital transformation. (Sakovich, 2024). By 2024, it is projected that the total value of the software development market will reach $698.80 billion, as seen in Figure 5.

Figure 5. Software market value worldwide per year. (Statista, 2023)



Statista also predicts that the worldwide number of software developers will hit 28.7 million this year, 2024, as seen in Figure 6. (Sakovich, 2024)

Figure 6. Number of software developers worldwide in 2018 and 2024. (Statista, 2023)



The number of software developers worldwide is increasing at a steady rate year after year. This means that more junior developers are entering the field.

3.1.3 AI Impact on the Job Market

"Robots will take our jobs. We had a better plan now before it is too late. " (Guardian, 2018)

Artificial intelligence is transforming how humans live and work. It is being used across various industries to improve a wide variety of things, such as operations and efficiency. Industries like healthcare, I.T., and manufacturing, among many others, are being affected. However, as AI becomes more prevalent, concerns are growing about its possible impact on the job market, particularly for new programmers entering the market. (Varadi, 2023)

One of the most widely discussed and feared aspects of AI is how this technology will affect the job market, but more peculiarly, the potential for AI to put people out of work. Computers do not get tired, they do not have excuses to turn up late for work, they do not argue or complain, and perhaps more importantly, computers do not need to get paid to complete their tasks. This raises concerns among the employees and interests of the employers. (Wooldridge, 2020, p.264)

Headlines talking about AI making you redundant have been around quite significant for the past few years, however, this is nothing new. Large-scale automation of human labor goes back to the Industrial Revolution. This revolution introduced a new way in which goods were manufactured, and this revolution was prompted by a range of different technological

advancements during the time. This led to a considerable change in work for much of the population during the time. (Wooldridge, 2020, p.264-265)

A similar pattern can be observed when examining how technology has transformed industries. A key technological advancement was the microprocessor. Microprocessors make today's computer technology possible. Before this technology, computers were large, expensive, and relatively unreliable. Microprocessors made computers cheap, fast, reliable, and smaller, and that made possible the automation of countless jobs. (Wooldridge, 2020, p.267).

While the effects of automation were adverse in some parts of the world, it is essential to note that the net effect that microprocessors had, economically speaking, was positive. (Wooldridge, 2020, p.267-268)

Emerging technologies create new opportunities for businesses, services, and wealth creation. This was the case for microprocessors. More jobs and wealth were created compared to those that were destroyed. (Wooldridge, 2020, p.268)

However, Michael Wooldridge (2020) says that the effect that AI will have will be on the nature of work itself. He says that most workers will not be replaced by this technology. People will start using this technology to make workers better and more efficient in the tasks they perform. For example, the tractor did not replace farmer workers, on the contrary, it made them better and more efficient.

AI will change the nature of work. However, it is still being determined whether these changes will be as dramatic as those of the Industrial Revolution or if these changes will be more modest. This debate started in 2013 with a report called "The Future of Employment" written by Carl Frey and Michael Osborne. The report stated that around 47% of jobs in the United States were susceptible to automation by AI and related technologies in the relatively near future. (Wooldridge, 2020, p.268)

As discussed above, with the microprocessor's innovation created between the years 1970s and early 1980s, it's important to note that the creation of this technology led to a wave of automation, and the public debate caused by that automation at the time is very similar to the debate of the implications of AI nowadays. (Wooldridge, 2020, p.271-272).

While AI is a heavily debated topic and a significant factor when it comes to changing the landscape of work, it is by no means the only factor it may even not be the most important one. For starters, globalization has not reached the end of its journey, and before it reaches the end, it will continue to change the world. Then there are computers, computers themselves are constantly evolving and changing for good. These machines keep getting cheaper, smaller, and more interconnected, and these innovations will continue to change the world and how humans live and work. Furthermore, there is still an ever-changing social, economic, and political landscape. All of these factors will have effects on employment and plenty of other things, and those effects will be noticeable alongside those of AI (Wooldridge, 2020, p.273-274)

As discussed above, the thesis explored the rapidly evolving landscape of artificial intelligence and its impact on job opportunities, it is essential to look back into the past and understand the historical context and lessons learned from previous technological revolutions. While headlines may induce fear about AI replacing human labor, it is essential to recognize that technological advancements have led to shifts in employment rather than job displacement. The Industrial Revolution, for example, brought significant changes in the nature of work but ultimately led to the creation of more jobs and wealth than it displaced. Similarly, the creation of microprocessors revolutionized computing, making tasks more efficient without eliminating jobs outright.

The introduction of AI will undoubtedly transform the nature of work, but the extent of its impact remains uncertain. While some jobs may become automated, AI has the potential to improve human capabilities and make tasks more efficient, much like the tractor did for farmers, as discussed above. The debate surrounding AI's implications is very similar to past discussions about technological advancements, highlighting the need for careful consideration of its potential effects.

AI is just one of many factors affecting the future of work. Globalization continued advancements in computing technology, and shifting social, economic, and political landscapes will all play significant roles in determining employment trends. It is essential to approach the integration of AI into the workforce with a balanced perspective, recognizing both its potential benefits and challenges.

3.1.4 AI Impact on Junior Developer Roles

Recently, over the past couple of years, the number of businesses using AI has grown exponentially. AI is transforming the nature of traditional tech roles, including junior developers. (SkillReactor, 2023). This includes tasks performed by junior developers. There is a significant concern regarding AI reducing the job prospects for entry-level programmers (Varadi, 2023). In this subchapter, the thesis will navigate through the current rise of AI and how will this affect junior developers. Challenges, opportunities, and job prospects, among other things, will be discussed.

Automated systems are becoming increasingly popular, and advanced AI is already taking over human jobs in various areas. For instance, tasks such as coding, testing, and debugging that were previously handled by junior programmers are now being automated with generative AI in some cases. This shift towards automation may lead to a decrease in entry-level programming positions. (Varadi, 2023).

The ability of AI to automate repetitive tasks and streamline workflows is changing the way developers work, and it is introducing new valuable skills and opportunities for the industry. However, it is okay. AI also brings new opportunities for junior developers, like the example mentioned in the subchapter above, tractors did not replace farmers, but instead helped farmers and provided more areas of opportunities for farmers. Junior developers now have the opportunity to master new tools and techniques related to AI, such as machine learning and data science, among others. By doing this, junior developers can position themselves with an advantage within the industry. (Mansoor, 2023).

One way in which AI is changing the workflow of junior developers is automation. Automation is a crucial component of modern developer workflows. It has revolutionized the way tasks are completed, freeing up valuable time for developers to focus on other relevant things. Automation has brought about significant changes in developers' daily tasks. From automated testing and deployment to continuous integration and delivery. (Mansoor, 2023)

Although adopting automation can bring many benefits, developers need to adapt to new technologies and workflows. This often requires them to learn new skills and approaches. Furthermore, developers must continue to improve their technical expertise to create and maintain the automation tools required for their work, as well as stay up to date with the current frameworks and technologies. (Mansoor, 2023)

Now, the rise of AI, automation, and the possibilities created by generative AI can significantly impact the skillsets required for junior developers to succeed in the tech industry. As mentioned above, tasks are becoming automated, and due to this, more than technical skills are needed to stand out in today's crowded job market. (Mansoor, 2023)

Programming languages and frameworks remain a crucial factor for junior developers, however, developers need to have various soft skills, adaptability, collaboration skills, and good communication skills. AI and the opportunities this technology created require developers to adapt quickly to new technologies and ways of work. Also, new technical skills are in high demand for both junior and senior developers. Demand for skills such as machine learning and data analysis is growing, and junior developers who can demonstrate an understanding of these skills will have a competitive advantage. (Mansoor, 2023)

Nowadays, it is a genuine concern for junior developers that employment prospects will change negatively due to the rise of AI. However, working with AI does present certain benefits. One of the primary advantages for junior engineers is the improved efficiency they can gain from working with generative AI like ChatGPT. By utilizing AI-powered systems to automate tasks such as coding, testing, and debugging, junior developers can spend more time on other projects. Consequently, junior developers may become more productive and satisfied in their work. (Varadi, 2023).

The increasing use of AI is making it harder for beginner programmers to find job opportunities in certain areas. However, there are also new opportunities opening up in fields such as machine learning, data science, and software engineering. Junior developers must stay up-to-date with the latest AI advancements and work hard to improve their skills if they want to remain competitive in the job market. (Varadi, 2023)

Retail, banking, and manufacturing industries are among the businesses that have started using AI to automate jobs that were previously done by less experienced programmers. Retailers are increasingly relying on chatbots powered by AI to handle customer support inquiries, which could decrease the demand for entry-level programmers in this field. Similarly, the financial sector is using AI to analyze data and detect signs of fraud, which could also result in a decreased demand for entry-level programmers in this sector. (Varadi, 2023)

As AI technology continues to advance, it can solve increasingly more complicated problems. As a junior developer, if you learn to work with AI, you can be an indispensable asset to any

company that requires AI-based solutions for large and intricate projects. Proficiency in AI as a junior developer can also improve your career prospects. Businesses are actively seeking junior developers with AI skills to develop and implement AI-based solutions. Some big companies are already putting AI to work alongside their junior developers. Google, Facebook, and Amazon are some of the companies that are heavily investing in AI to enhance their goods and services. They are also hiring junior developers with AI experience. (Varadi, 2023)

4 Chatbots and Artificial Intelligence Generated Code.

In this chapter, the thesis explores chatbots and AI-generated code. Chatbots, powered by artificial intelligence, have revolutionized interactions with technology. Their capabilities, history and their opportunity as tools will be explored.

4.1 History of Chatbots

It is important to note that not all chatbots are AI chatbots. An AI chatbot is a software program that understands human language, whether it is spoken or written. This is achieved through a technology called Natural Language Processing (N.L.P.), which allows the AI chatbot to determine the user's intent and respond accordingly. In simpler terms, an AI chatbot can figure out what the user is asking for and provide the best possible answer. With the help of machine learning, this AI technology can mimic human-like traits by being able to solve problems, think creatively, and reason independently. (Ventoniemi, n.d)

Chatbots have been around for quite some time already, but it was not until recently that this technology gained real popularity among users and businesses alike. (Ina, 2022). In the 1960's, M.I.T. professor Joseph Weizenbaum created E.L.I.Z.A. This chatbot was the first ever to be developed. (Ida, 2022). This program attempted to simulate the experience of speaking to a therapist. However, this technology was less versatile than similar technologies currently existing. E.L.I.Z.A. relied on natural language understanding, reacted to keywords, and then replied to the user. Joseph Weizenbaum wrote in his 1966 paper the following, "some subjects have been tough to convince that E.L.I.Z.A. (with its present script) is not human." (Thorbecke, 2022)

Figure 7. Conversation between the ELIZA program and humans. (Szymczak, 2017).

Another chatbot was created by American psychiatrist Kenneth Colby in 1972. PARRY is a natural language program that resembles the thinking of an individual. PARRY was designed to imitate a patient with schizophrenia while also attempting to simulate the disease. (Ina, 2022)

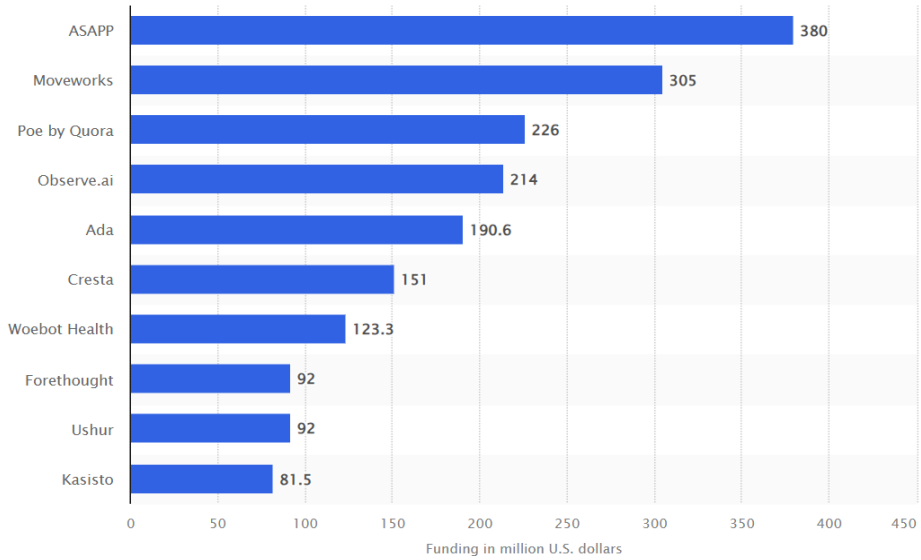
PARRY uses a complex system of assumptions, attributions, and "emotional responses" that are triggered by changing weights assigned to verbal inputs. To test its effectiveness, PARRY underwent a variation of the Turing test. During the test, which took place in the early 1970s, human interrogators interacted with the program via a remote keyboard and were only slightly more accurate than random in their ability to distinguish PARRY from an actual irrational individual. (Ina, 2022)

Many chatbots have been created, such as Jabberwacky, which aimed to simulate a natural human conversation entertainingly. Or chatbot Dr. Sbaitso. Dr. Sbaitso is one of the earliest chatbots incorporating AI and it is recognized for being fully voice-operated. (Ina, 2022). However, there is a chatbot that is considered head and shoulders above any other AI chatbots during this time. Artificial Linguistic Computer Entity, also known as A.L.I.C.E., is a language-processing chatbot and robotic program. A.L.I.C.E. engages in electronic chat with humans. (Rouse, 2023)

In 1995, while attending Lehigh University in Bethlehem, Pennsylvania, Richard Wallace developed A.L.I.C.E., known initially as Alicebot, because it was first run on a computer by the name of Alice. The A.L.I.C.E. program uses the artificial intelligence markup language

(A.I.M.L.) XML schema to define conversation rules. A.L.I.C.E. was initially written in Java in 1998, and Wallace published an A.I.M.L. specification in 2001. Other developers created free and open versions of A.L.I.C.E. in various programming languages as well as foreign languages. (Rouse, 2023)

Figure 8. Leading chatbot/conversational AI startups worldwide in 2023, by funding raised. (Statista, 2023)



Several decades later, the market is now filled with chatbots that vary in quality and have different use cases, and they are developed by tech companies, banks, airlines, and more. In many ways, Weizenbaum's story predicted the excitement and confusion that still surrounds AI-powered chatbots today. Some people are still amazed by the ability of a program to converse with humans, leading them to believe that the machine is somehow closer to being human. (Thorbecke, 2022)

4.2 Generative AI

Generative AI refers to deep learning models that can generate high-quality content, such as text and images, based on the data they were trained on. (Martineau, 2023)

According to Martineau (2023). "Artificial intelligence has gone through many cycles of hype, but even to skeptics, the release of ChatGPT seems to mark a turning point."

In the past, generative AI has made significant advancements in computer vision, transforming selfies into Renaissance-style portraits and aged faces. However, the focus has now shifted to natural language processing, where large language models can generate content on any topic. Generative models have also learned to understand the grammar of software code, molecules, natural images, and other data types. Not only that, but the applications of this technology are growing larger every day, and the possibilities are just starting to be explored. (Martineau, 2023)

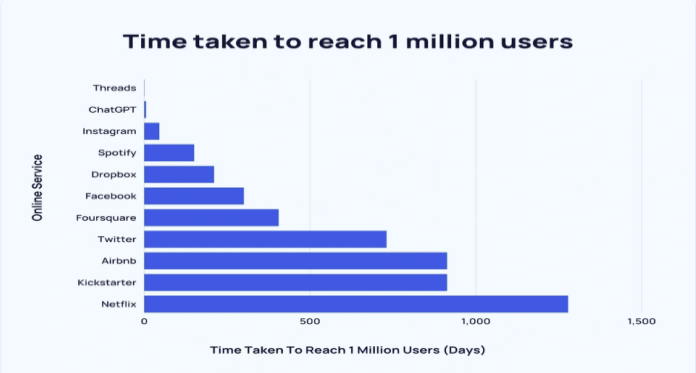
4.2.1 ChatGPT

ChatGPT, a language processing tool based on GPT-3.5 architecture, was introduced in November 2022 and has gained immense popularity due to its human-like responses. Researchers are optimistic about its potential to revolutionize various fields, including code generation and software development. However, there are concerns about the reliability and quality of the code generated by ChatGPT. Its widespread use could pose potential risks that have yet to be explored. (Liu, Le-Cong, Widyasari, Tantithamthavorn, Li, Le, & Lo, 2024, p. 1)

The interaction between users and ChatGPT feels different compared to other software. Previous software using natural-language processing was limited to short exchanges and predetermined responses. However, ChatGPT can generate its own content and continue the dialogue for a more extended period. (Baker, 2023)

ChatGPT, similar to other machine-learning (ML) and deep learning (DL) models, gains knowledge through exposure to patterns in extensive training datasets. This enables it to identify and recognize similar patterns in other datasets. However, ChatGPT's learning process is different from that of humans. It understands and reacts to information based on its pattern recognition abilities, rather than thinking or learning like humans. Currently, ChatGPT supports 95 languages. It also knows several programming languages, such as Python and JavaScript. (Baker, 2023)

Figure 9. The time it took to reach 1 million users. (Duarte, 2024)



Research conducted by UBS said that ChatGPT is currently the "fastest-growing consumer application in history." To compare this growth with other popular applications, TikTok took nine months to reach 100 million monthly users, while Instagram took two and a half years to reach such a number. As seen in Figure 9, ChatGPT surpassed 1 million users in just five days. (Gordon, 2023)

4.2.2 Leveraging Generative AI as a Tool

The launch of generative AI tools such as GPT3, GPT4, and Google Bard, among others, took the industry to a rapid transformation. Throughout this thesis, the impact of this technology has been analyzed theoretically, considering the job prospects and how it can affect junior developers both positively and negatively. It is known that generative AI will create new software-related roles as well as replace already existing ones. (Sauvola, Tarkoma, Klemettinen, Riekki, & Doermann, 2024, p. 2)

Generative AI has redefined industries, from roles to tools, processes, and operations. The use of GPT-powered P.O.C.s (proof of concepts) and services has encouraged innovations, and we see them coming almost every week. For example, many game developers and graphics creation communities have integrated multiplatform capabilities, such as AutoGPT models, to make the most of G.P.T. Generative AI, which is seen as a solution to optimize resources and improve software development quality. By freeing high-level skills, it reduces costs and improves productivity for many corporations. (Sauvola, Tarkoma, Klemettinen, Riekki, & Doermann, 2024, p. 2)

As mentioned throughout this thesis, the demand for software development roles and innovations has been increasing year after year. This has led to the automation of work and

processes, and due to this, the development and maintenance of software-based applications are experiencing a transformation thanks to the introduction of generative AI models. For example, software developers are adopting co-pilots, interpreters, and private automated G.P.T. models to help them optimize their tools and workflows. Moreover, machine learning and N.L.P. (Natural Language Processing) is helping software developers with code queries, production stages, code review, code testing, configuration, and optimization. (Sauvola, Tarkoma, Klemettinen, Riekki, & Doermann, 2024, p. 2-3)

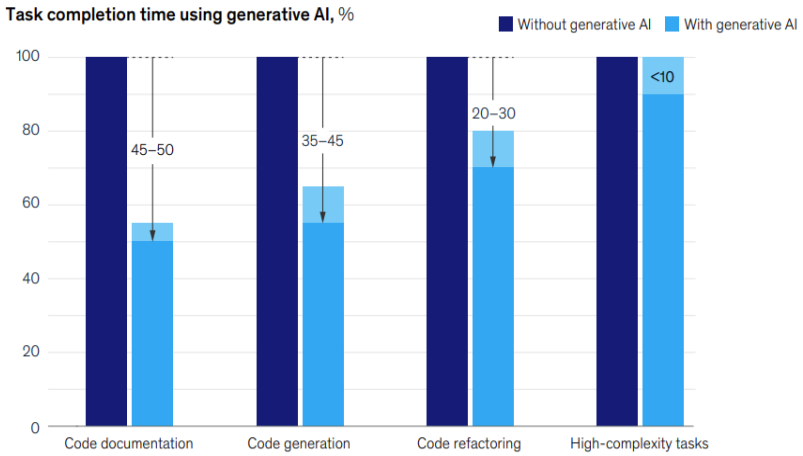
Generative AI is not the only type of AI that affects software development. AI like supervised learning, unsupervised learning, and reinforcement learning play a part in the software development industry. Previous AI solutions have been offered through these other types of AI, however, generative AI is currently viewed as a game changer within the industry due to its capabilities. (Sauvola, Tarkoma, Klemettinen, Riekki, & Doermann, 2024, p. 3)

Several AI tools are available to assist with code discovery, generation, optimization, documentation, testing, debugging, and simulation of different execution models. However, in the future, AI will be used in a mix of roles. For instance, the integration of supervised and unsupervised learning can dramatically reduce development times. An example of this would be the automated configuration of systems that learn and adapt to preferences. The use of these types of AI collaboratively can drive productivity improvements. (Sauvola, Tarkoma, Klemettinen, Riekki, & Doermann, 2024, p. 3)

Recent studies show that software developers can complete their coding tasks up to twice as fast if working alongside generative AI like ChatGPT. (Deniz, Gnanasambandam, Harrysson, Hussin, & Srivastava, 2023, p. 1)

As seen below in Figure 10, data suggests that developers completed everyday tasks faster with the use of generative AI tools. Tasks such as documenting code functionality for maintainability can be completed as fast as half the time compared to not using generative AI tools. Writing new code can also be completed in nearly half the time, and code refactoring can be done in nearly two-thirds of the time. Of course, these increases in task completion speed are significant, which means that software developers will be more productive.

Figure 10. Software developer tasks speed comparison using generative AI and without generative AI (Deniz, Gnanasambandam, Harrysson, Hussin, & Srivastava, 2023, p. 2)



It is important to note that results are only beneficial considering tasks that a developer considers to be 'easy.' As seen above in Figure 10, the difference in speed between using generative AI or not with high-complexity tasks is minimal. This can be due to several factors. For example, a developer needs to gain programming knowledge for a specific framework. According to the study conducted by McKinsey Digital, similar results can be seen among junior developers with less than a year of experience. In some cases, junior developers took around 7-10 percent longer with tools than without them. (Deniz, Gnanasambandam, Harrysson, Hussin, & Srivastava, 2023, p. 2)

The use of AI-assisted tools did not negatively affect the quality of the code generated for the sake of speed. In the study, when developers worked together with the tool, the code quality was slightly better in terms of bugs, maintainability, and readability, which is essential for reusability. However, developer feedback indicates that they actively collaborated with the tools to achieve the desired code quality. This suggests that AI technology is best employed to improve, rather than replace, developers. Also, it is important that to maintain code quality, developers must understand the factors that constitute quality code and guide the tool to produce the appropriate output. (Deniz, Gnanasambandam, Harrysson, Hussin, & Srivastava, 2023, p. 2-3)

In the study conducted by McKinsey Digital, generative AI-based tools increased productivity in four main areas.

With the use of generative AI, developers can save up quite some time with routine, easy, or repetitive tasks such as finishing coding statements as the developer types and documentation of the code based on the developer's requests. As already mentioned, by doing so, developers have more time to focus on other complex tasks or challenges to increase performance and develop new software capabilities. (Deniz, Gnanasambandam, Harrysson, Hussin, & Srivastava, 2023, p. 3)

Get a first draft of new code: Developers who do not know exactly what to do next can use generative AI to request coding suggestions. Developers might not know how to fix a specific problem or need some ideas on how to tackle such a problem and generative AI can be an excellent tool for such occasions. Those developers who used generative AI reported that they managed to overcome the blank screen and started coding much faster with the help of the code suggestions given. (Deniz, Gnanasambandam, Harrysson, Hussin, & Srivastava, 2023, p.4)

Updates to existing code: If needed, developers can use generative AI tools to make changes to already existing code faster. For example, developers can spend less time adapting code from a library or improving already existing code. (Deniz, Gnanasambandam, Harrysson, Hussin, & Srivastava, 2023, p. 4)

Facing new challenges: It is common for developers to encounter unfamiliar codebases, frameworks, etc. With generative AI tools, developers can rapidly understand and work with unfamiliar codebases, frameworks, or languages if necessary. Moreover, when facing challenges, they can use these tools as an alternative option compared to seeking help from a colleague. (Deniz, Gnanasambandam, Harrysson, Hussin, & Srivastava, 2023, p. 4)

It is important to note that a good understanding and skills in programming languages are only some of the critical things required for software development. Writing code needs a solid foundation in logic, problem-solving, and analytical thinking. Acquiring these skills is like building blocks, where learning to code is the first step, just like arithmetic and algebra are fundamental to advanced mathematics. (Duranton, 2024)

As attractive and functional generative AI within the software development industry can be, there is only so much this tool can do. The human touch within the software development industry remains crucial and, in some instances, unreplaceable. Generative AI, when it comes to AI-generated code is, as discussed, a great tool for both junior and senior developers, however, the tool could be better.

The developer must examine the code for either bugs or errors. In some cases, the code provided by generative AI can give incorrect recommendations or even give the developer code with errors. (Deniz, Gnanasambandam, Harrysson, Hussin, & Srivastava, 2023, p. 5). It is very important to keep an eye out for the code given by generative AI Even with the advanced capabilities of these tools, the AI may occasionally provide the wrong code. It is the developer's job to keep an eye out for these types of errors and act accordingly to fix the issue.

Although AI may help to write more code, it does not mean it would be of good quality. The developer should evaluate the quality, or else the output may end up with poorly written code that is difficult to read and lacks structure. (Duranton, 2024)

Now, while generative AI tools know a lot about coding, the AI will never truly know the specific needs of a given project. This knowledge is of great importance when coding to make sure the final software developed meets the standards set by the company, can be integrated with other applications, and meets security standards. (Deniz, Gnanasambandam, Harrysson, Hussin, & Srivastava, 2023, p. 5)

For the time being, AI-assisted coding will liberate programmers from tedious and repetitive tasks. This will allow them to concentrate on more innovative and complex problem-solving. In the future, coders may need to change their approach from simply mastering specific programming languages to comprehending fundamental programming concepts and acquiring the skills to work collaboratively with AI systems. (Duranton, 2024)

5 Research Methodologies

In this chapter, the methodologies employed in conducting the practical aspects of this research are outlined. There will be a focus on understanding the experiences of junior developers using generative AI as well as testing and analyzing how effective generative AI is with several coding tasks. The research methodologies utilized include experimental research with LeetCode problems and a survey aimed at HAMK's computer applications students.

5.1 Experimental Research: LeetCode Problems

Experimental research uses LeetCode problems to assess the efficacy of generative AI in coding tasks.

LeetCode is a widely known platform within the software development industry. Leetcode is a platform offering tech skills and interview preparations for either students or senior developers. With LeetCode, users can practice and take on a wide variety of programming problems. LeetCode provide users with the opportunity to practice problems covering various programming topics, including software engineering, computer science, data structures, algorithms, system design, SQL, dynamic programming, decision trees, machine learning, and artificial intelligence. (Girardin. M, 2023)

In this study, ChatGPT AI is tasked with solving a selection of LeetCode problems across all three difficulty levels, easy, medium, and hard. The code generated by ChatGPT is then evaluated for correctness, time, and prompts needed to achieve the correct answer. By analyzing ChatGPT's performance on these problems, insights into its effectiveness in assisting junior developers with coding tasks are gained as well as how effective and optimal this tool is.

5.1.1 Experimental Research Design and Problem Selection

The platform used for this test will be ChatGPT v3.5. For a comprehensive evaluation, problems with difficulty of easy, medium, and hard will be tested within the category of algorithms. JavaScript and Python programming languages will be used.

5.1.2 Experiment Procedure

Two problems from each difficulty level will be selected to ensure a balanced representation of challenges. After the problems have been selected, the AI will be tasked to generate a solution to the selected problem in the two programming languages already specified. Important to note that the prompt given to ChatGPT will be the same as the prompt that LeetCode gives to the user. Furthermore, speed and accuracy for each problem solved will be evaluated.

5.2 Survey Methodology

In addition to experimental research, a survey is conducted to gather firsthand experiences and perceptions of junior developers regarding generative AI in coding-related tasks. The survey aims to understand how junior developers interact with generative AI tools, their perceived benefits and challenges, and the overall impact on their coding skills and career trajectory. Questions in the survey cover topics such as frequency of AI tool usage, satisfaction with generated code, concerns about reliability and accuracy, and perceived effects on skill development and job opportunities. The survey data provides qualitative insights complementing the quantitative findings from the experimental research.

5.2.1 Survey Design

The survey consists of nine questions designed to gather information about participants' academic year, programming experience, usage of AI-generated code, perceptions of its quality, perceived benefits, problem-solving methods, and encountered challenges.

5.2.2 Survey Distribution

The survey for this study was distributed to HAMK's computer applications students through the Webropol platform. The survey was advertised on Yammer, specifically within the computer applications group. This approach ensured that the survey was visible to the relevant students.

5.3 Data Collection and Analysis

Data collection involves administering LeetCode problems to ChatGPT AI and distributing the survey questionnaire to a targeted sample of junior developers. The responses obtained from both sources are then analyzed using appropriate statistical and qualitative analysis techniques. For the LeetCode experiment, quantitative metrics such as accuracy rates, and execution times. In contrast, the survey responses will be analyzed to identify recurring patterns, themes, and sentiments expressed by junior developers. The integration of both quantitative and qualitative data enhances the comprehensiveness and depth of the research findings.

6 Survey Findings and Analysis

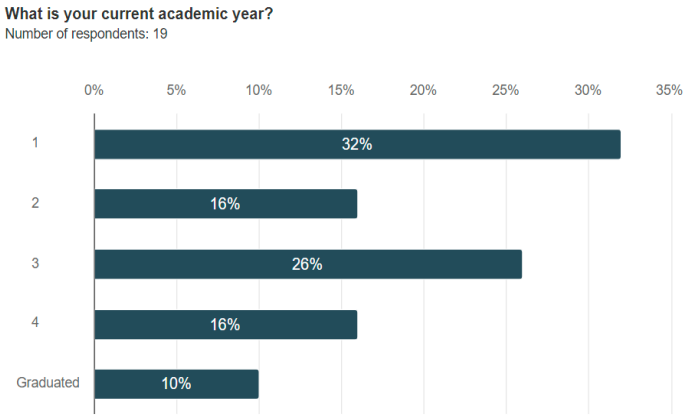
This chapter presents and analyzes the results of the survey conducted to gather information from students in the Degree Programme in Computer Applications at HAMK on their thoughts about generative AI in the software development world. The survey's purpose is to understand the perceptions, experiences, and expectations of students regarding the use of generative AI tools in their programming practices.

The survey consisted of a series of questions designed to capture various aspects of student opinions, including their familiarity with generative AI, the benefits and challenges, and the potential impact on their learning and future careers. The complete list of survey questions is provided in the Appendix.

6.1 Background of Students

To provide context for the survey findings, it is important to understand the background of the students who participated in the survey. The first question asked HAMK's computer application students to indicate their current academic year.

Figure 11. Academic Year of Students.

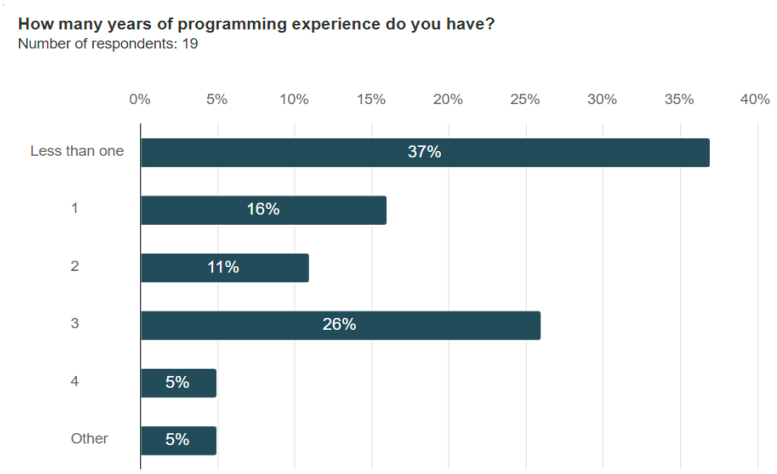


The survey results indicate the distribution of respondents across different academic years. Out of the 19 respondents, the largest group, are first year students with a total of 32% of the respondents being first year. Third-year students form the second largest group at 26%. Both second-year and fourth-year students each represent 16% of the respondents. Additionally, 10% of the respondents have already graduated. There is a wide representation from all

academic years, with a significant number of participants from the early and middle stages of their academic progress.

The second question asked participants to indicate their years of experience in programming. This question helps to contextualize the familiarity and proficiency with the programming of the respondents, which may influence their perspectives on the use of generative AI.

Figure 12. Years of experience in programming.

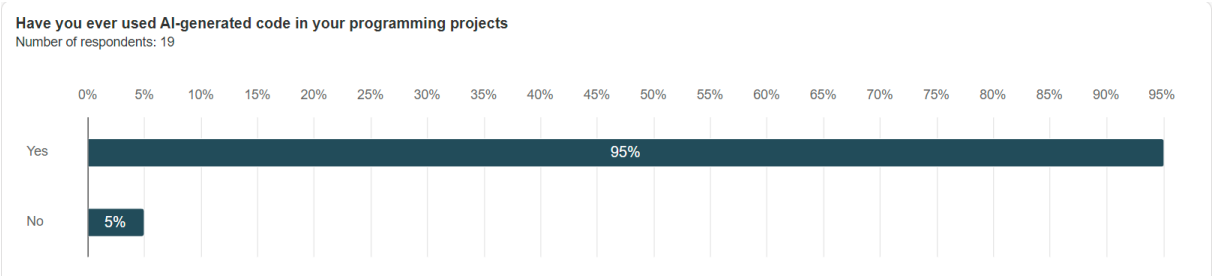


The survey results reveal a varied range of programming experiences among the respondents. A total of 37% of respondents have less than a year of experience. This is followed by those with three years of experience, making up 26% of the respondents. Participants with one year and two years of experience represent 16% and 11%, respectively. A smaller proportion of respondents, each accounting for 5%, indicated having four years of experience or falling into the "Other" category. This answer highlights that a significant portion of the respondents are relatively new to programming, with a smaller but notable group having more substantial experience.

6.2 Use of Generative AI tools

The first question in this section asked respondents whether they have used AI-generated code in their programming projects. This question aims to have an idea of the prevalence of generative AI tool usage among students.

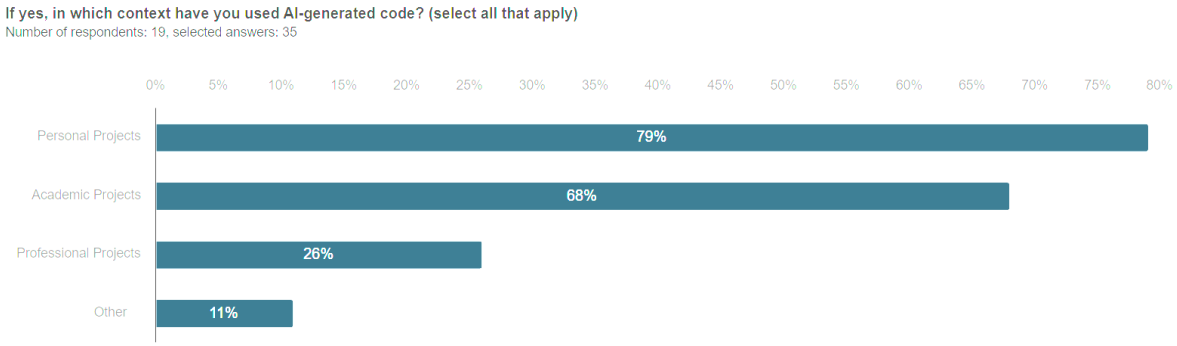
Figure 13. Use of AI-generated code.



It is important to note that the 5% that said that they have not used such tools represents only 1 respondent. All but one respondent has used generative AI. This indicates a high level of engagement with this technology.

The next question asked those who have used such code to specify the contexts in which they have done so. Respondents were allowed to select several options including personal projects, academic projects, professional projects, and others. This will provide insight into the various ways in which generative AI tools are being integrated into their programming activities.

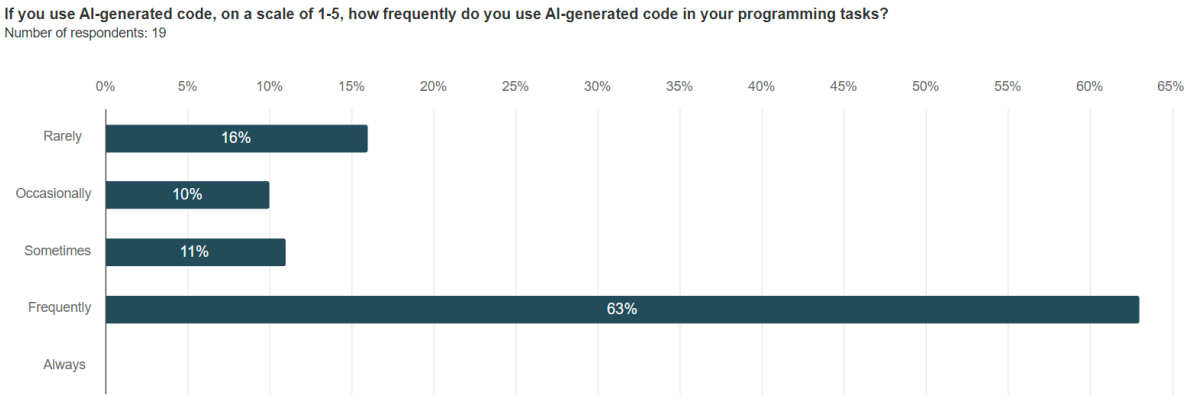
Figure 14. Context of usage.



The survey results show that 79% of respondents use AI-generated code in personal projects and 68% in academic projects. Only 26% have used it in professional projects, and 11% in other contexts. This indicates that AI-generated code is primarily utilized in personal and academic settings.

The second question of this section asked respondents to rate on a scale of 1 to 5 the frequency of their usage of AI-generated code. This question aims to capture how often students integrate AI-generated code into their programming activities.

Figure 15. Frequency of usage.

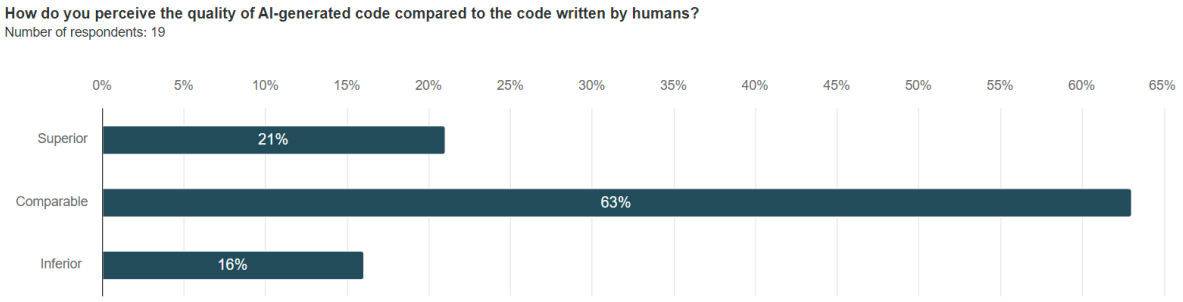


The survey results indicate that 63% of respondents, frequently use AI-generated code in their programming tasks. A smaller proportion use it sometimes (11%), occasionally (10%), and rarely (16%). None of the respondents reported always using AI-generated code. This suggests that while AI-generated code is commonly used by students, its use is not yet fully employed across all programming tasks.

6.3 Quality Perceived of AI-Generated Code

The first question on this subchapter asked respondents to compare the code generated by AI to that of the code written by humans. This question aims to capture the respondents's views on the effectiveness, reliability, and overall quality of AI-generated code, providing insight into how these tools are regarded in comparison to traditional human coding practices.

Figure 16. Perceived quality AI vs human code.

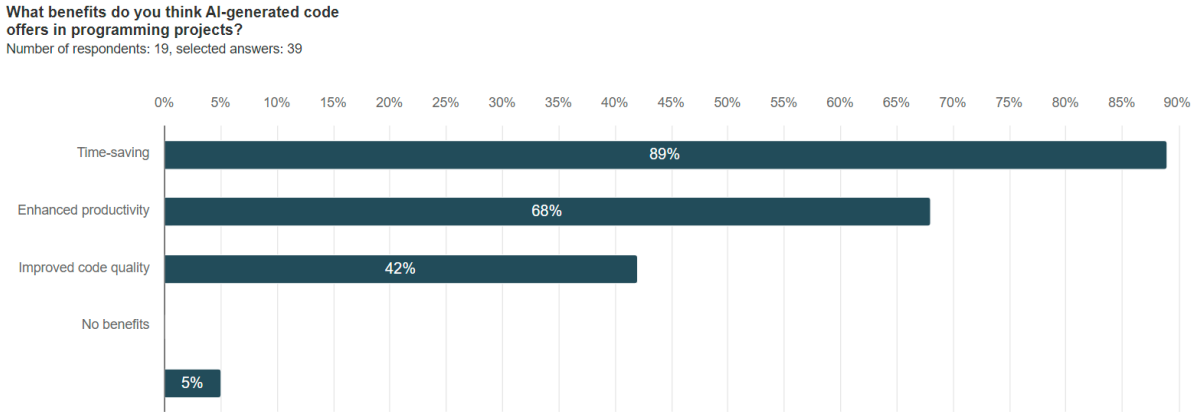


A vast majority of respondents, 63%, indicated that AI-generated code is comparable to that of human-written code. However, 21% of respondents indicated that AI-generated code is superior. Only 16% of respondents signaled that AI-generated code is inferior when compared to human code.

6.4 Perceived Benefits of AI-generated Code.

The first question on this subchapter asked respondents to state the perceived benefits AI-generated code has. The goal of this question is to explore what specific benefits programmers believe AI-generated code offers to their day-to-day programming tasks. This can give some context on how AI is slowly transforming the development work landscape on aspects like productivity, creativity, and efficiency.

Figure 17. Perceived benefits.



AI is transforming the development field. As seen throughout the thesis and the survey questions, developers are leaning towards using this technology more often as time goes by.

The goal of this question was to explore the perceived benefits this technology brings to the table when it comes to programming tasks. A vast percentage of respondents (89%) indicated that this technology is primarily beneficial for saving time. Referring to Figure 10, and Figure 17, it is clear that this technology helps developers with time in a wide range of programming tasks. 68% of respondents believe that AI enhances productivity, 42% perceive an improvement in code quality and a small minority of 5% indicated the ‘other’ field with no additional answers on the text field.

6.5 Code Problem Methods

This subchapter explores the methods developers rely on when facing coding problems or errors. It aims to understand the primary resources or strategies used by developers to find solutions or explanations to their coding queries. For this, respondents were asked to specify the methods they use to seek solutions or explanations.

Figure 18. Methods used for code queries.

	n	Percent
Utilizing AI chatbots or virtual assistants designed to assist with coding-related queries.	12	63.2%
Googling the problem or error to find the solutions or explanations from online forums, documentation, or tutorials	17	89.5%
Other	1	5.3%

Most respondents, 89.5%, rely on Googling the issue to find solutions or explanations from online forums, documentation, or tutorials. Additionally, 63.2% of respondents utilize AI chatbots or virtual assistants designed to help with coding-related queries, showing a significant adoption of AI tools in problem-solving. A small fraction, 5.3%, mentioned using other methods, specifically the GDB debugger.

6.6 AI-generated Challenges and Limitations

The last subchapter for this section goes over the challenges and limitations that developers have encountered when using AI-generated code. This open-ended question aims to uncover specific issues and drawbacks that users face, providing a comprehensive understanding of the potential downsides and areas of opportunity in AI-assisted coding.

Figure 19. Challenges and limitations for AI-generated code.

Responses
AI doesn't fully understand what I'm asking, keeps making the same mistakes if the prompt isn't right.
While working on a personal coding project using react, most of the times I asked the AI for code or assistance, the input given wasn't the best. It helps but it isn't the best tool to use for high complexity problems or tasks
Most of the times I have used AI for my coding tasks, the code I get is never correct at first, extra inputs or explanations are needed and even so, its not always correct either way.
Sometimes, the code provides is not adequate. It is still up to the developer to see those faults and know how to fix them
Plenty of extra prompts
Sometimes, the code provided gives errors or doen't work propertly
The AI doesn't know to detail the parameters of the project being worked on
the chatbot's code is mostly used in the specific context of your question, and doesn't really take into account the wider scope of the whole project, also the fact that it is mostly prediction based means it will always have an answer no matter what and when there is an error in the code generated it leads it to write code in circles instead of addressing the root cause of the error
-
LLM gets the general idea but many many small typos. Also it's hard to take the problems from your head and give them to AI because often you do not know the problem before you get to know it. And when you know it and can define the problem, you can pretty much write your own code. Often I feel I need to go and design and write my own code to build large projects. AI can help solve some problems, but if we talk about bigger projects, it helps a lot when you really know the code (i.e. you've thought and written it yourself). I find it challenging to have some AI to write code and then I need to understand what it did and how it thought to solve the problem. You cannot really build the project so well in your head if you don't do the thinking. But of course you can ask the AI for help on how to implement for instance an A* Algorithm, that can help. Hope this helps!
Not all the code is working as wanted, sometimes propting correct code from (atleast free AI) can take sometime, and atleast on mybehalf I've noticed that it passivates to learn coding
The AI might not understand the context of the project.
- You have to keep the conversation as short as possible since (at least the free version of) chatgpt becomes confused after a while - I think it works best in English, so people who don't use English maybe can't use its full potential. - Programmers usually benefit more from AI bots since they know how computers operate, while "normal" people get frustrated with a "dumb robot". - I have a pretty positive outlook on AI, it has the potential for amazing things. But also terrible things, like its use in war and weapons. Countries should limit its use in wartime and surveillance of citizens but also strive towards using it for good like medicine, innovation and maybe in an ideal situation even getting away from capitalism if it would take all our jobs.
A lot of times, the code generated by the AI haves some mistakes.
using AI-generated code could be like lottery, sometimes it works after few attempts (rarely). But in most of the times it is time consuming and could be more confusing.
It is usually incorrect, but leads in righ direction.
It's difficult to debug
None
Generative AI isnt perfect, output given is most of the times inaccurate or wrong.

Users report that the code provided by AI often requires further refinement and debugging, as it is frequently incorrect or incomplete. Additionally, AI-generated solutions can sometimes be overly generic, lacking the contextual understanding necessary for complex or specific tasks. The need for additional prompts and explanations to guide the AI indicates that it does not always produce optimal or efficient code on the first try. Some users find the process of correcting and refining AI-generated code time-consuming and potentially more confusing than helpful. One respondent indicated a concern regarding the learning potential of coding. The respondent said it passivates code learning, leading to relying mainly on AI for coding problems thus minimizing problem-solving skills and code comprehension.

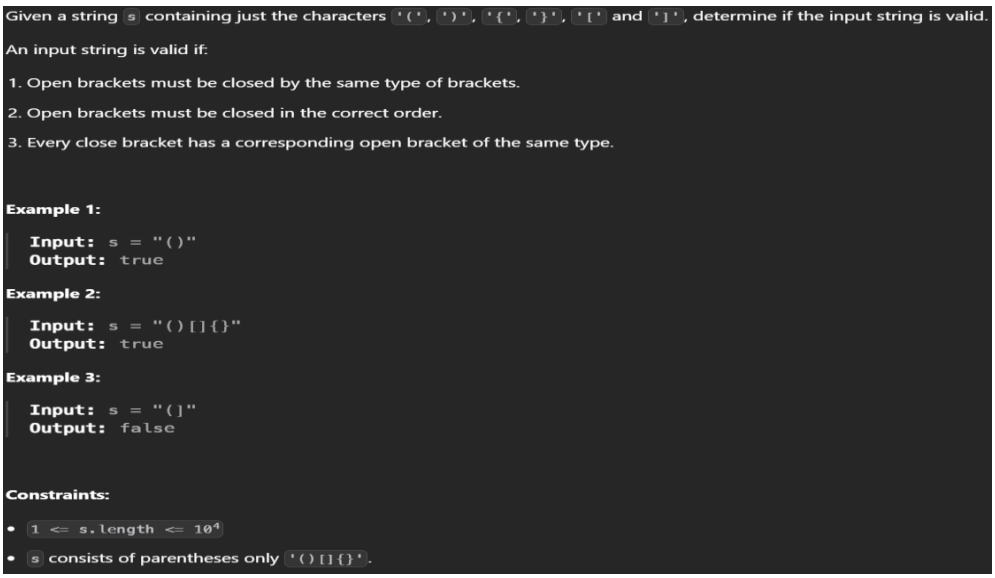
7 Experimental Research

In this chapter, the practical implementation of the thesis research will be conducted. The effectiveness of generative AI in solving coding problems will be assessed and unlike previous chapters, which were mainly focused on theoretical groundwork, this section focuses on hands-on experimentation to provide insights into generative AI within coding.

7.1 LeetCode Problems: Easy Difficulty

The first task within this difficulty range is called Valid Parentheses. For this problem, ChatGPT is required to determine if the input string is valid considering that a string s contains the characters '(', ')', '{', '}', '[' and ']'. Figure 20 shows the problem more in detail. JavaScript will be the programming language used.

Figure 20. LeetCode problem 20. Valid Parentheses problem description. (LeetCode, n.d)



With ChatGPT's version 3.5, users can now share screenshots within the chat. To provide the problem prompt to ChatGPT, a screenshot of the problem (as shown in Figure 20) will be shared, and the programming language specified. This approach will be consistently applied to all subsequent problems.

Figure 21 shows the code generated by ChatGPT to answer problem number 20.

Figure 21. Final answer generated by ChatGPT to problem number 20.

```javascript
function isValid(s) {
    const stack = [];
    const mapping = {
        ')': '(',
        '}': '{',
        ']': '['
    };

    for (let char of s) {
        if (char in mapping) {
            const topElement = stack.pop() || '#';
            if (mapping[char] !== topElement) {
                return false;
            }
        } else {
            stack.push(char);
        }
    }

    return stack.length === 0;
}

// Test cases
console.log(isValid("()"));         // Output: true
console.log(isValid("()[]{}"));     // Output: true
console.log(isValid("(]"));         // Output: false
console.log(isValid("([)]"));       // Output: false
console.log(isValid("{[]}"));       // Output: true
```

For the analysis of problem 1, the code was successfully generated by the AI in 6.40 seconds, demonstrating prompt and efficient response time. The generated code provided a correct solution to the problem statement without requiring any additional prompts or clarifications. It is important to note that the problem's acceptance rate is 42.4%

The second problem within this difficulty range is problem number 3079. Find the Sum of Encrypted Integers. Figure 22 shows the description and instructions of the problem. Python will be the programming language used for this problem.

Figure 22. LeetCode problem 3079. Find the Sum of Encrypted Integers description. (LeetCode, n.d)

You are given an integer array `nums` containing **positive** integers. We define a function `encrypt` such that `encrypt(x)` replaces **every** digit in `x` with the **largest** digit in `x`. For example, `encrypt(523) = 555` and `encrypt(213) = 333`.

Return *the **sum** of encrypted elements.*

Example 1:

> **Input:** `nums = [1,2,3]`
>
> **Output:** 6
>
> **Explanation:** The encrypted elements are `[1,2,3]`. The sum of encrypted elements is `1 + 2 + 3 == 6`.

Example 2:

> **Input:** `nums = [10,21,31]`
>
> **Output:** 66
>
> **Explanation:** The encrypted elements are `[11,22,33]`. The sum of encrypted elements is `11 + 22 + 33 == 66`.

Constraints:

- `1 <= nums.length <= 50`
- `1 <= nums[i] <= 1000`

The prompt given to the AI will be the same as seen in Figure 22.

For the analysis of problem 2, the code was successfully generated by the AI in 8.29 seconds. The generated code provided a correct solution to the problem statement without requiring any additional prompts or clarifications. It is important to note that the problem's acceptance rate is 73.4%

7.2 LeetCode Problems: Medium Difficulty

The first task is problem number 3121 called Count the Number of Special Characters II. For this problem, a string **word** is given. A letter c is called special if it appears both in lowercase and uppercase in the string **word**, and every lowercase occurrence of c appears before the first uppercase in the **word.** ChatGPT's task is to return the number of special letters in the word. Python will be the programming language used.

Figure 23. LeetCode problem 3121 description and instructions. (LeetCode, n.d)

You are given a string `word`. A letter `c` is called **special** if it appears **both** in lowercase and uppercase in `word`, and **every** lowercase occurrence of `c` appears before the **first** uppercase occurrence of `c`.

Return the number of **special** letters in `word`.

Example 1:

Input: `word = "aaAbcBC"`

Output: 3

Explanation:

The special characters are `'a'`, `'b'`, and `'c'`.

Example 2:

Input: `word = "abc"`

Output: `0`

Explanation:

There are no special characters in `word`.

Example 3:

Input: `word = "AbBCab"`

Output: `0`

Explanation:

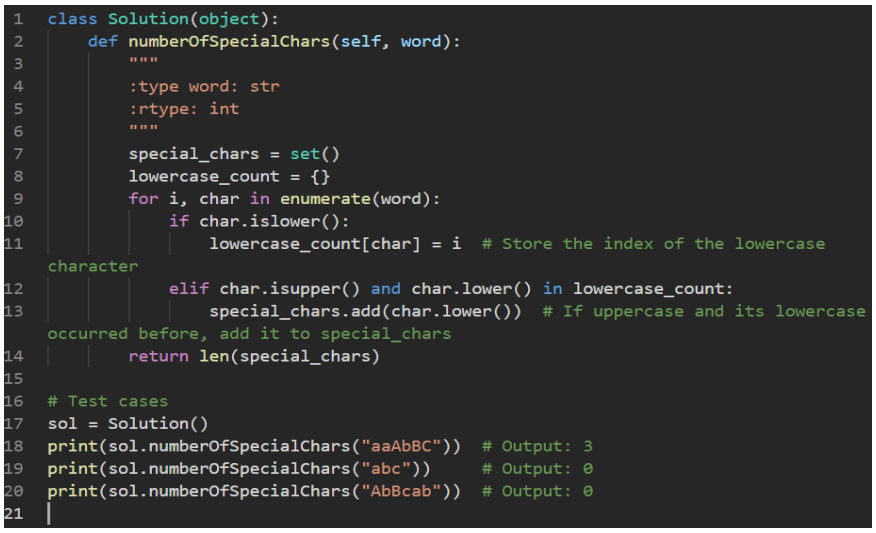
There are no special characters in `word`.

Constraints:

- `1 <= word.length <= 2 * 10^5`
- `word` consists of only lowercase and uppercase English letters.

As mentioned above, the prompt given to the AI will be the same as in Figure 23 above.

Figure 24. The first answer generated by ChatGPT to problem 3121.

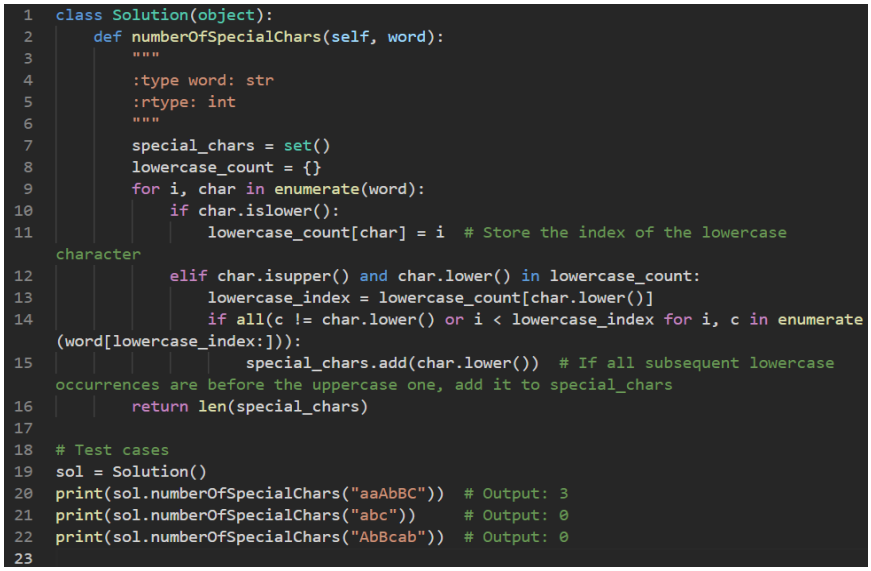


```python
class Solution(object):
    def numberOfSpecialChars(self, word):
        """
        :type word: str
        :rtype: int
        """
        special_chars = set()
        lowercase_count = {}
        for i, char in enumerate(word):
            if char.islower():
                lowercase_count[char] = i  # Store the index of the lowercase character
            elif char.isupper() and char.lower() in lowercase_count:
                special_chars.add(char.lower())  # If uppercase and its lowercase occurred before, add it to special_chars
        return len(special_chars)

# Test cases
sol = Solution()
print(sol.numberOfSpecialChars("aaAbBC"))  # Output: 3
print(sol.numberOfSpecialChars("abc"))     # Output: 0
print(sol.numberOfSpecialChars("AbBcab"))  # Output: 0
```

For problem 2, while the AI correctly generated code that produced the expected output for the first two cases, it encountered an error in the third case. The output for case 3 did not match the expected result, indicating a failure to handle certain edge cases or conditions within the problem statement. Further prompts to the AI will be given to fix the problem.

Figure 25. Final answer generated by ChatGPT to problem 3121.



```python
class Solution(object):
    def numberOfSpecialChars(self, word):
        """
        :type word: str
        :rtype: int
        """
        special_chars = set()
        lowercase_count = {}
        for i, char in enumerate(word):
            if char.islower():
                lowercase_count[char] = i  # Store the index of the lowercase character
            elif char.isupper() and char.lower() in lowercase_count:
                lowercase_index = lowercase_count[char.lower()]
                if all(c != char.lower() or i < lowercase_index for i, c in enumerate(word[lowercase_index:])):
                    special_chars.add(char.lower())  # If all subsequent lowercase occurrences are before the uppercase one, add it to special_chars
        return len(special_chars)

# Test cases
sol = Solution()
print(sol.numberOfSpecialChars("aaAbBC"))  # Output: 3
print(sol.numberOfSpecialChars("abc"))     # Output: 0
print(sol.numberOfSpecialChars("AbBcab"))  # Output: 0
```

For problem 2, the AI successfully produced the correct code as seen in Figure 25 after receiving four additional prompts and 1 minute and 19 seconds. During this process, three incorrect answers were initially provided by the AI, followed by a runtime error. After

receiving the necessary guidance through the prompts, the AI eventually generated the correct solution. The acceptance rate for this problem is 39.9%

The second task is problem number 80. Remove Duplicates from Sorted Array II. Figure 26 shows the problem more in detail. JavaScript will be the programming language used.

Figure 26. Description and instructions for problem 80. Remove duplicates from sorted array II. (LeetCode, n.d)

Given an integer array `nums` sorted in **non-decreasing order**, remove some duplicates in-place such that each unique element appears **at most twice**. The **relative order** of the elements should be kept the **same**.

Since it is impossible to change the length of the array in some languages, you must instead have the result be placed in the **first part** of the array `nums`. More formally, if there are `k` elements after removing the duplicates, then the first `k` elements of `nums` should hold the final result. It does not matter what you leave beyond the first `k` elements.

Return `k` *after placing the final result in the first* `k` *slots of* `nums`.

Do **not** allocate extra space for another array. You must do this by **modifying the input array** in-place with O(1) extra memory.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

```
Input: nums = [1,1,1,2,2,3]
Output: 5, nums = [1,1,2,2,3,_]
Explanation: Your function should return k = 5, with the first five elements of nums being 1, 1, 2, 2 and 3 respectively.
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Example 2:

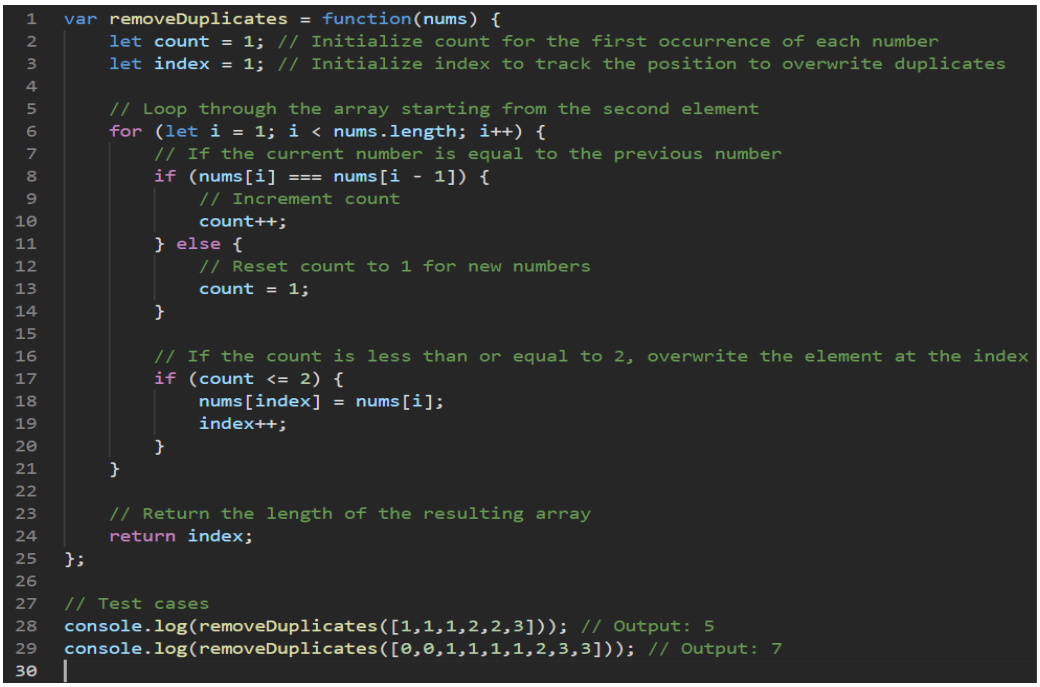
```
Input: nums = [0,0,1,1,1,1,2,3,3]
Output: 7, nums = [0,0,1,1,2,3,3,_,_]
Explanation: Your function should return k = 7, with the first seven elements of nums being 0, 0, 1, 1, 2, 3 and 3 respectively.
It does not matter what you leave beyond the returned k (hence they are underscores).
```

Constraints:

- `1 <= nums.length <= 3 * 10^4`
- `-10^4 <= nums[i] <= 10^4`
- `nums` is sorted in **non-decreasing** order.

The prompt given to ChatGPT will be the same as shown in Figure 26.

Figure 27. Final answer generated by ChatGPT to problem 80.



```javascript
var removeDuplicates = function(nums) {
    let count = 1; // Initialize count for the first occurrence of each number
    let index = 1; // Initialize index to track the position to overwrite duplicates

    // Loop through the array starting from the second element
    for (let i = 1; i < nums.length; i++) {
        // If the current number is equal to the previous number
        if (nums[i] === nums[i - 1]) {
            // Increment count
            count++;
        } else {
            // Reset count to 1 for new numbers
            count = 1;
        }

        // If the count is less than or equal to 2, overwrite the element at the index
        if (count <= 2) {
            nums[index] = nums[i];
            index++;
        }
    }

    // Return the length of the resulting array
    return index;
};

// Test cases
console.log(removeDuplicates([1,1,1,2,2,3])); // Output: 5
console.log(removeDuplicates([0,0,1,1,1,1,2,3,3])); // Output: 7
```

The AI completed the second problem for medium difficulty. It took approximately 14.88 seconds to generate the solution, and no further prompts were needed to obtain a correct answer.

7.3 LeetCode Hard Difficulty

The first problem for the hard difficulty is problem number 10. Regular Expression Matching. Figure 28 contains the details and the instructions for the problem. The problem will be solved in JavaScript.

Figure 28. Description and instructions for problem 10. Regular Expression Matching. (LeetCode, n.d)

Given an input string `s` and a pattern `p`, implement regular expression matching with support for `'.'` and `'*'` where:

- `'.'` Matches any single character.
- `'*'` Matches zero or more of the preceding element.

The matching should cover the **entire** input string (not partial).

Example 1:

```
Input: s = "aa", p = "a"
Output: false
Explanation: "a" does not match the entire string "aa".
```

Example 2:

```
Input: s = "aa", p = "a*"
Output: true
Explanation: '*' means zero or more of the preceding element, 'a'. Therefore, by repeating 'a' once, it
becomes "aa".
```

Example 3:

```
Input: s = "ab", p = ".*"
Output: true
Explanation: ".*" means "zero or more (*) of any character (.)".
```

Constraints:

- `1 <= s.length <= 20`
- `1 <= p.length <= 20`
- `s` contains only lowercase English letters.
- `p` contains only lowercase English letters, `'.'`, and `'*'`.
- It is guaranteed for each appearance of the character `'*'`, there will be a previous valid character to match.

The prompt given to ChatGPT will be the same as seen in Figure 28.

Figure 29. Final answer generated by ChatGPT to problem 10.

```javascript
var isMatch = function(s, p) {
    const dp = Array(s.length + 1).fill(false).map(() => Array(p.length + 1).fill(false));
    dp[0][0] = true;

    for (let j = 1; j <= p.length; j++) {
        if (p[j - 1] === '*') {
            dp[0][j] = dp[0][j - 2];
        }
    }

    for (let i = 1; i <= s.length; i++) {
        for (let j = 1; j <= p.length; j++) {
            if (s[i - 1] === p[j - 1] || p[j - 1] === '.') {
                dp[i][j] = dp[i - 1][j - 1];
            } else if (p[j - 1] === '*') {
                dp[i][j] = dp[i][j - 2] || (dp[i - 1][j] && (s[i - 1] === p[j - 2] || p[j - 2] === '.'));
            }
        }
    }

    return dp[s.length][p.length];
};

```

The AI completed the first problem for hard difficulty. It took approximately 5.37 seconds to generate the solution, and no further prompts were needed to obtain a correct answer. The acceptance rate for this problem is 28.1%.

The second problem for the hard difficulty is problem number 2945. Find the Maximum Non-decreasing Array Length. Figure 30 will contain details and instructions for the problem. The programming language used will be Python.

Figure 30. Description and instructions for problem 2945. Find the Maximum Non-decreasing Array Length. (LeetCode, n.d)

You are given a **0-indexed** integer array `nums`.

You can perform any number of operations, where each operation involves selecting a **subarray** of the array and replacing it with the **sum** of its elements. For example, if the given array is `[1,3,5,6]` and you select subarray `[3,5]` the array will convert to `[1,8,6]`.

Return *the **maximum** length of a **non-decreasing** array that can be made after applying operations.*

A **subarray** is a contiguous **non-empty** sequence of elements within an array.

Example 1:

> **Input:** nums = [5,2,2]
> **Output:** 1
> **Explanation:** This array with length 3 is not non-decreasing.
> We have two ways to make the array length two.
> First, choosing subarray [2,2] converts the array to [5,4].
> Second, choosing subarray [5,2] converts the array to [7,2].
> In these two ways the array is not non-decreasing.
> And if we choose subarray [5,2,2] and replace it with [9] it becomes non-decreasing.
> So the answer is 1.

Example 2:

> **Input:** nums = [1,2,3,4]
> **Output:** 4
> **Explanation:** The array is non-decreasing. So the answer is 4.

Example 3:

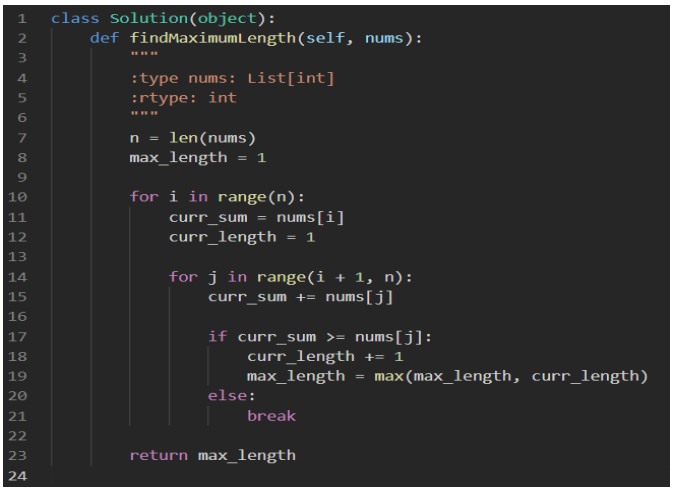
> **Input:** nums = [4,3,2,6]
> **Output:** 3
> **Explanation:** Replacing [3,2] with [5] converts the given array to [4,5,6] that is non-decreasing.
> Because the given array is not non-decreasing, the maximum possible answer is 3.

Constraints:

- `1 <= nums.length <= 10^5`
- `1 <= nums[i] <= 10^5`

Figure 30 will serve as the prompt given to ChatGPT.

Figure 31. Answer generated by ChatGPT to problem 2945.



```python
class Solution(object):
    def findMaximumLength(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
        n = len(nums)
        max_length = 1

        for i in range(n):
            curr_sum = nums[i]
            curr_length = 1

            for j in range(i + 1, n):
                curr_sum += nums[j]

                if curr_sum >= nums[j]:
                    curr_length += 1
                    max_length = max(max_length, curr_length)
                else:
                    break

        return max_length

```

For problem 2, while the AI correctly generated code that produced the expected output for case 2, wrong outputs were produced in cases 1 and 3. Further prompts to the AI will be given to fix the problem.

Despite the attempts and ten consecutive prompts, the AI was unable to produce the correct solution to the given problem. In each attempt, the provided outputs were incorrect, indicating a consistent inability to understand and address the problem's requirements accurately. The acceptance rate for the problem is 15.5%.

8 Results and Analysis

This chapter presents the results of the experimental research conducted to assess the performance of generative AI, specifically ChatGPT v3.5, the theoretical research on how can this technology be used as an empowering tool, as well as the survey findings.

8.1 Leveraging AI as a Tool

The theoretical research demonstrates that AI, particularly generative AI like ChatGPT, can significantly improve developer’s productivity by streamlining routine tasks and providing coding suggestions. The findings suggest that AI tools can reduce task completion time for simple coding activities such as code documentation, code generation, and code refactoring by nearly 50%. This time saved allows developers to focus on more complex and innovative tasks, improving overall software development quality and experience.

However, several factors affect the AI's effectiveness. Among these factors, the complexity of the tasks is one of the most important ones. As seen in Figure 10, AI's effectiveness on time saved was almost null when a high-complexity task was at play.

Furthermore, while AI tools can assist in generating and refining code, it is crucial that developers actively collaborate with the tool to ensure the quality and relevance of the output. Figure 17 shows that HAMK's computer applications students indicate that this tool has the already mentioned benefits, however, none of the students indicated that this tool has no benefit whatsoever. Also, Figure 19 shows the limitations and challenges encountered with this tool and the main consensus is that this tool is far from perfect. Therefore, AI is best leveraged as a supportive tool that enhances productivity while still requiring the developer's expertise to maintain high standards in coding and problem-solving.

8.2 LeetCode Problems

The results of the LeetCode experiment demonstrated that AI tools, in this case, ChatGPT, can be effective at solving programming problems, however, its success varied across different difficulty levels as well as other factors such as framework utilized, prompt give, release date of problem, etc.

Both problems in the easy difficulty category were solved successfully by ChatGPT on the first attempt. This suggests that the AI performed well when presented with coding problems of lower complexity, demonstrating its capability to generate accurate solutions without the need for additional guidance.

In the medium difficulty category, one problem was solved on the first attempt, indicating the AI's proficiency in handling moderately challenging coding tasks. However, the other problem required four prompts before arriving at the correct solution. This suggests that while the AI can effectively solve medium-level problems, its performance may vary depending on the complexity and nature of the problem as well as the clarity of the prompts given.

For the hard difficulty category, ChatGPT solved one problem immediately. This demonstrates its potential to solve complex coding challenges effectively. However, the second problem in this category was unsolvable by the AI. This shows its limitations in dealing with highly complex or abstract problems. These results highlight the influence of factors such as problem complexity and the time of introduction, suggesting that ChatGPT may struggle with certain advanced tasks, particularly those that are newly introduced or require deep understanding beyond pattern recognition. As illustrated in Figure 30, the problem to be solved is problem number 2945, indicating that it was released relatively recently. Given that the experiment was conducted after ChatGPT's last training update, which only includes data up to January 2022, ChatGPT likely has more comprehensive knowledge of older problems than of newer ones.

It is important to know that when the initial code generated by the AI for a LeetCode problem was incorrect or did not produce the expected results, the process involved providing the AI with specific feedback to attempt to guide it toward a correct solution. The AI was informed that the solution was incorrect, and the console output from LeetCode, which included error messages or test case failures, was provided as feedback. This output helped the AI to reassess the problem and refine the code given. The AI would then generate a revised code, which was tested again on LeetCode. This iterative process was repeated until the AI produced a code solution that successfully passed all test cases.

The experiment indicates that ChatGPT is effective at solving easy problems, and consistently providing accurate solutions on the first attempt. For medium difficulty problems, while the AI remains capable, its effectiveness diminishes slightly, as evidenced by the need for multiple prompts in one of the problems. For the hard problems, ChatGPT is less effective, although it succeeded in solving one hard difficulty problem, it was unable to solve

another, highlighting significant limitations. These findings suggest that while AI tools like ChatGPT are good aids in programming, their effectiveness can be affected by plenty of variables.

8.3 Survey

Students revealed that they are starting to incorporate this technology into their programming tasks in various types of contexts. Figure 13 shows that 95% of students have used this technology within their programming tasks, and Figure 15 shows that 63% of them use it frequently. Only 5% representing one student have not used this technology for their programming tasks. Figure 14 shows how students use this technology in several ways. 79% of students use it for personal matters, 68% of students for academic matters, 26% for professional matters, and 11% for other matters.

Figure 16 shows that 63% of students perceive the code generated by AI as comparable to that of a human and students also Figure 17 reveals the perceived benefits of AI-generated code. Students indicated that it is time saving, enhances productivity, improves code quality, and other benefits, however, no student indicated that this technology has no benefits whatsoever.

The traditional troubleshooting method is still preferred, Figure 18 shows how most students still use Google or online documentation and forums for their coding queries, however, the use of AI assistants is increasing. Moreover, students indicated the challenges and limitations this technology has. Students indicated that the code generated is buggy, requires extra prompting for a correct answer, lacks contextual knowledge, and human oversight is heavily needed among other concerns.

8.4 Results Summary

The thesis explored the effectiveness of generative AI, particularly ChatGPT, in various aspects of software development, revealing both significant advantages if used and notable limitations. The thesis found that AI tools can reduce the time required for simple coding tasks by nearly 50%, allowing developers to focus on more complex aspects of their work. However, the effectiveness of AI decreases as task complexity increases. Human oversight is needed to ensure code quality and relevance.

The experiment conducted using LeetCode problems demonstrated that ChatGPT performs well with simpler tasks, solving both of the problems on the first try. However as difficulty increases, its performance varies. For the medium difficulty problems, one of the problems required multiple prompts to reach the correct solution. In the hard difficulty category, the AI successfully solved one problem but failed to solve another. Factors affect the effectiveness of the AI to successfully solve problems that extend beyond its training data

The survey results indicate widespread adoption of AI tools among students, 95% indicated that they have used this technology for their programming tasks and 63% frequently use AI-generated code. Most respondents view AI-generated code as comparable to human-written code, though traditional troubleshooting methods remain prevalent. The key benefits identified by the students include time saving, enhanced productivity, and improved code quality. Moreover, none of the students indicated that this tool has no benefits. However, students indicated plenty of challenges and limitations. Among the most important ones include further refinement of the code generated and lack of contextual understanding of the project or task being worked on.

9 Conclusions

The primary goal of this thesis was to investigate the effectiveness and impact of generative AI in the context of software development, specifically focusing on its capabilities in generating code, its potential as a tool for junior developers, and its integration and perception among students within the industry. The thesis employed experimental research in which the AI was tested with some leetcode problems, a survey was used to get insights on computer application students with their experience and sentiment of AI within the AI-generated code field, and wide theoretical research was conducted to understand how is this technology affecting the software development industry and junior developers alike.

To achieve this, the following research questions were answered.

- How effective is generative AI in generating code, ranging from solving easy to complex programming tasks?
- In what ways can AI-generated code be leveraged as an empowering tool for junior developers?
- How do students use, integrate, and perceive AI generative tools within the software development industry?

As for the first research question, ChatGPT performs well with easy coding problems, solving them on the first attempt. For medium-difficulty problems, it shows variable results, successfully solving one immediately but needing multiple prompts for another. In hard difficulty, it can solve complex problems but struggles with highly intricate ones, indicating limitations with the most challenging tasks. The time in which the leetcode problem was introduced and the complexity of the problem affect the outcome of the answer.

For the second research question, AI-generated code can help junior developers by automating routine tasks like documentation, code generation, and refactoring, allowing them to focus on more complex aspects of software development. Tools such as ChatGPT offer coding suggestions and debugging assistance, helping junior developers overcome challenges and understand unfamiliar codebases faster. However, it is essential for developers to critically evaluate the AI-generated code to ensure quality and correctness. This synergy enhances learning and skill development if used correctly, making generative AI a valuable tool for junior developers.

Lastly, the survey provided valuable insight regarding how students use generative AI for their coding tasks. As seen on the survey, a total of 95% of respondents have used some

sort of generative AI for their coding tasks, students are starting to use this technology more frequently in various contexts such as academic projects, personal projects, professional projects, etc. However, there are real concerns regarding the quality of the code generated. Most respondents indicated limitations when it comes to this technology.

9.1 Self-assessment

This thesis is composed of three main pillars. A theoretical research whose goal is to answer how AI can be used as an empowering tool, experimental research that measures how effective AI is in solving programming problems, and a survey whose goal is to get information on how software development students use this technology.

The qualitative aspect of the theoretical research provided a comprehensive understanding of how AI can be an empowering tool for developers. This was done by analyzing existing studies and data to further understand the benefits this tool provides to developers. The thesis explores the benefits this tool provides, alongside its limitations.

The experimental component was grounded in quantitative data, with performance metrics collected from the AI's effectiveness in solving a range of programming problems on LeetCode. By selecting problems across the easy, medium, and hard difficulty levels, the experiment generated data that illustrated the AI's problem-solving capabilities, providing measurable evidence of its utility in certain scenarios, and its futility in other scenarios.

Lastly, the survey gathered both qualitative and quantitative data from HAMK's computer application students. Quantitatively, it provided data on how frequently students use AI tools, what types of tasks they use them for, and how they perceive their efficacy. Qualitatively, the survey gathers information on the sentiments, concerns, and expectations of students regarding AI-generated code.

Looking ahead to further development of the thesis, there is always room for improvement and this thesis is no exception. For example, in the experimental research, the experiment was limited to only two programming problems tested; having a bigger sample size would have provided more comprehensive and statistically significant data. Additionally, the role of prompting in AI performance is a crucial factor that could have been explored further. Effective prompting, specificity, and context can impact AI's ability to solve coding problems. Including more context on best prompting practices and experimenting with different strategies would have offered deeper insights into optimizing AI-assisted coding.

A similar observation can be made regarding the survey conducted as part of this thesis. The sample size consisted of 19 students. While the survey provided valuable results, a larger number of respondents would have significantly enhanced and enriched the findings. Increasing the sample size could have yielded more comprehensive data, allowing for more precise conclusions.

9.2 Learnings

Throughout the thesis, I explored how artificial intelligence will impact junior developers and the software development industry. Before I started to write the thesis, I, like many other junior developers, was concerned that this technology would make me obsolete when it come to work life. But now, after finishing the thesis process and gathering all of the results, my mindset towards this technology is different.

First of all, thanks to some theoretical research and the experiment conducted with leetcode problems, I now have a deeper understanding of generative AI’s capabilities within the software development industry, more specifically in code generation. This thesis provided a clearer view of both the strengths and limitations of this technology. I now know that this technology is far from perfect and this tool will not replace all of the skills a junior developer brings to the table.

Although this tool is far from perfect, this does not mean it is bad. Thanks to the theoretical research and a survey conducted, I learned that this tool can boost a developer's efficiency when it comes to their day-to-day coding tasks. If utilized correctly, this tool can empower developers to be more efficient.

But what I learned the most after finishing the thesis process, is that software development is more than just programming. There are a lot of building blocks that make software development what it is and AI cannot replace such building blocks. For example, there are skills such as critical thinking, problem-solving skills, and creativity. Artificial intelligence does not know a client's needs, artificial intelligence does not know how to collaborate within a team. Human creativity and intuition are irreplaceable when it comes to outside-the-box thinking or ethical decision making when facing challenges. These aspects of software development require a certain level of judgment, empathy, collaboration, and foresight that AI, despite its recent advances, cannot replicate.

10 References

Aceves-Fernandez, M. A. (2018). *Artificial Intelligence: Emerging Trends and Applications.* IntechOpen. Retrieved March 27, 2024, from https://directory.doabooks.org/handle/20.500.12854/130071

Antony, B. (2023). *AI Chatbots: Threat or Opportunity?* Retrieved April 4, 2024, from https://www.mdpi.com/2227-9709/10/2/49

Anyoha, R. (2017). *The History of Artificial Intelligence*. Retrieved April 1, 2024, from https://sitn.hms.harvard.edu/flash/2017/history-artificial-intelligence/

Baker, P. (2023). *ChatGPT For Dummies.* Retrieved April 7, 2024, from https://learning.oreilly.com/library/view/chatgpt-for-dummies/9781394204632/c00.xhtml#h2-1

Ben, G. (2014). Artificial General Intelligence: Concept, State of the Art, and Future Prospects. *Journal of Artificial Intelligence, 5*(1). Retrieved April 1, 2024, from https://sciendo.com/article/10.2478/jagi-2014-0001

Bernd Carsten, S., Doris, S., & Rowena, R. (2023). *Ethics of Artificial Intelligence: Case Studies and Options for Addressing Ethical Challenges.* Springer Nature. Retrieved March 29, 2024, from https://directory.doabooks.org/handle/20.500.12854/93981

Britannica. (2024, January 26). *Artificial Intelligence (AI): At a Glance*. Retrieved April 3, 2024, from https://www.britannica.com/topic/Artificial-Intelligence-AI-At-a-Glance-2235722

Carlos P, J. D. (2009). *asfsfsa.*

Clark, E. (2023). *Unveiling The Dark Side Of Artificial Intelligence In The Job Market*. Retrieved April 13, 2024, from Forbes: https://www.forbes.com/sites/elijahclark/2023/08/18/unveiling-the-dark-side-of-artificial-intelligence-in-the-job-market/?sh=3c102b836652

Codeacademy Team. (n.d.). *History of Chatbots*. Retrieved April 8, 2024, from Codeacademy: https://www.codecademy.com/article/history-of-chatbots

Dechand, S. (n.d.). *The Risks of AI-Generated Code*. Retrieved from Code Intelligence: https://www.code-intelligence.com/blog/risks-of-ai-generated-code#reliance-onai

Deniz, B., Gnanasambandam, C., Harrysson, M., Hussin, A., & Srivastava, S. (2023, June). Unleashing developer productivity with generative AI. *McKinsey Digital*, 7. Retrieved April 19, 2024, from https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/unleashing-developer-productivity-with-generative-ai#/

Duarte, F. (2024, March 27). *Number of ChatGPT Users (Apr 2024)*. Retrieved April 14, 2024, from Exploding Topics: https://explodingtopics.com/blog/chatgpt-users

Duranton, S. (2024, April 15). *Are Coders' Jobs At Risk? AI's Impact On The Future Of Programming*. Retrieved April 19, 2024, from Forbes: https://www.forbes.com/sites/sylvainduranton/2024/04/15/are-coders-jobs-at-risk-ais-impact-on-the-future-of-programming/?sh=51e940d373e5

Girardin, M. (2023, December 15). *What Is LeetCode?* Retrieved April 23, 2024, from Forage: https://www.theforage.com/blog/skills/what-is-leetcode

Gordon, C. (2023, February 2). *ChatGPT Is The Fastest Growing App In The History Of Web Applications*. Retrieved April 12, 2024, from Forbes: https://www.forbes.com/sites/cindygordon/2023/02/02/chatgpt-is-the-fastest-growing-ap-in-the-history-of-web-applications/?sh=2eb0a8d1678c

Gupta, S. (2023, May 1). *What happens to software jobs with the rise of Generative AI?* Retrieved April 16, 2024, from Linkedin: https://www.linkedin.com/pulse/what-happens-software-jobs-rise-generative-ai-sachin-gupta/

Gwerzman, D. (2023). *The Future of Junior Software Developers in the Age of AI/ML*. Retrieved April 12, 2024, from Medium: https://medium.com/@zps270/the-future-of-junior-software-developers-in-the-age-of-ai-ml-8532288e7055

Huma, S. (2008). A.L.I.C.E.: an ACE in Digitaland. *TripleC: Communication, Capitalisn & Critique, 4*(2), 284-292. Retrieved from https://doaj.org/article/3c2540a206274bec9b4f4917b7643770

IBM Data and AI Team. (2023, October 12). *Understanding the different types of artificial intelligence*. Retrieved April 2, 2024, from IBM: https://www.ibm.com/blog/understanding-the-different-types-of-artificial-intelligence/

Ina. (2022, March 15). *The History Of Chatbots – From ELIZA to ChatGPT*. Retrieved April 8, 2024, from Onlim: https://onlim.com/en/the-history-of-chatbots/

Jerry, K. (2016). *Artificial Intelligence: What Everyone Needs to Know.* Oxford University Press. Retrieved March 28, 2024, from https://books.google.fi/books?hl=en&lr=&id=wPvmDAAAQBAJ&oi=fnd&pg=PP1&dq=what+is+artificial+intelligence&ots=NzBDJAvuKe&sig=tBWbQaHJ89c6pYjAIb2_9bb7rSM&redir_esc=y#v=onepage&q=what%20is%20artificial%20intelligence&f=false

Kanade, V. (2022, March 14). *What Is Artificial Intelligence (AI)? Definition, Types, Goals, Challenges, and Trends in 2022*. Retrieved April 2, 2024, from SpiceWorks: https://www.spiceworks.com/tech/artificial-intelligence/articles/what-is-ai/

Laoyan, S. (2024, February 2). *What is Agile methodology? (A beginner's guide)*. Retrieved April 3, 2024, from Asana: https://asana.com/resources/agile-methodology

Lepri, B., Oliver, N., & Pentland, A. (2021). Ethical machines: The human-centric use of artificial intelligence. *IScience, 24*(3). Retrieved from https://doi.org/10.1016/j.isci.2021.102249

Maheshwari, R. (2023, April 3). *What Is Artificial Intelligence (AI) And How Does It Work?* Retrieved March 26, 2024, from Forbes Advisor: https://www.forbes.com/advisor/in/business/software/what-is-ai/

Mansoor, S. (2023, October 10). *The Impact of AI and Automation on Junior Developers' Job Prospects*. Retrieved April 8, 2024, from https://www.skillreactor.io/blog/the-impact-of-ai-and-automation-on-junior-developers-job-prospects/#:~:text=The%20rise%20of%20AI%20and,in%20a%20crowded%20job%20market.

Martineau, K. (2023, April 20). *What is generative AI?* Retrieved April 11, 2024, from IBM: https://research.ibm.com/blog/what-is-generative-AI

McGuire, M. (2024, March 24). *Top 4 software development methodologies*. Retrieved March 30, 2024, from Synopsys: https://www.synopsys.com/blogs/software-security/top-4-software-development-methodologies.html

Michael, H., & Andreas, K. (2019). A Brief History of Artificial Intelligence: On the Past, Present, and Future of Artificial Intelligence. *California Management Review, 61*(4), 5-185. Retrieved March 27, 2024, from https://doi-org.ezproxy.hamk.fi/10.1177/0008125619864925

Mollick, E. (2024). *Co-Intelligence : Living and Working with AI.* Penguin Publishing Group. Retrieved April 18, 2024, from https://ebookcentral-proquest-com.ezproxy.hamk.fi/lib/hamk-ebooks/reader.action?docID=30848006&ppg=90

Mucci, T., & Stryker, C. (2023, December 18). *What is artificial superintelligence?* Retrieved April 4, 2024, from IBM: https://www.ibm.com/topics/artificial-superintelligence

Osaba, E., Esther, V., Jesus L, L., & Ibai, L. (2021). *Artificial Intelligence Latest Advances, New Paradigms and Novel Applications.* IntechOpen. Retrieved March 28, 2024, from https://directory.doabooks.org/handle/20.500.12854/130994

Pickett, D. (2023, December 6). *AI Tools Aren't Replacing Junior Developers—They're Empowering Them*. Retrieved April 18, 2024, from Launch Academy: https://launchacademy.com/blog/ai-tools-empowering-junior-developers

Pinski, C. (2024, March 8). *The Future of the Software Development Industry: 2024 and Beyond*. Retrieved April 16, 2024, from Crispy Software Solutions: https://www.crispysoftwaresolutions.com/post/the-future-of-the-software-development-industry-2024-and-beyond

Rouse, M. (2023, November 8). *ALICE (Artificial Linguistic Computer Entity)*. Retrieved April 10, 2024, from https://www.techopedia.com/definition/380/artificial-linguistic-computer-entity-alice

Saeed, S., Jhanjhi, N., Naqvi, M., & Humayun, M. (2019). Analysis of Software Development Methodologies. *International Journal of Computing and Digital System, 8*(5), 445-460. Retrieved April 7, 2024, from 10.12785/ijcds/080502

Sakovich, N. (2024, April 8). *Top 10 Software Development Trends of 2024*. Retrieved April 12, 2024, from Sam Solutions: https://www.sam-solutions.com/blog/software-development-trends/

Sauvola, J., Tarkoma, S., Klemettinen, M., Riekki, J., & Doermann, D. (2024). Future of software development with generative AI. *Automated Software Engineering, 31*(1), 26. Retrieved April 16, 2024, from 10.1007/s10515-024-00426-z

Simplilearn. (2024). *What Is Software Development? Definition and Types*. Retrieved April 10, 2024, from Simplilearn: https://www.simplilearn.com/tutorials/programming-tutorial/what-is-software-development

Singh, T. (2019, August 29). *Software Ate The World, Now AI Is Eating Software*. Retrieved April 12, 2024, from Forbes: https://www.forbes.com/sites/cognitiveworld/2019/08/29/software-ate-the-world-now-ai-is-eating-software/?sh=684e605f5810

Statista. (2022, December). *Number of software developers worldwide in 2018 to 2024*. Retrieved April 10, 2024, from Statista: https://www.statista.com/statistics/627312/worldwide-developer-population/

Statista. (2023, December). *Application Development Software - Worldwide*. Retrieved April 15, 2024, from Statista: https://www.statista.com/outlook/tmo/software/application-development-software/worldwide#revenue

Statista. (2023, December). *Software - Worldwide*. Retrieved April 13, 2024, from Statista: https://www.statista.com/outlook/tmo/software/worldwide#revenue

Szymczak, A. (2017, January 3). *Introduction to chatbots in healthcare*. Retrieved April 9, 2024, from Infermedica: https://infermedica.com/blog/articles/introduction-to-chatbots-in-healthcare

The Lancet Digital Health. (2023). ChatGPT: friend or foe? *The Lancet. Digital health, 5*(3), 102. Retrieved from 10.1016/S2589-7500(23)00023-7

Thomas, W. (2022). *Product Development within Artificial Intelligence, Ethics and Legal Risk: Exemplary for Safe Autonomous Vehicles.* Springer Nature. Retrieved April 2, 2024, from https://directory.doabooks.org/handle/20.500.12854/81669

Thorbecke, C. (2022, August 20). *Chatbots: A long and complicated history*. Retrieved April 9, 2024, from CNN Business: https://edition.cnn.com/2022/08/20/tech/chatbot-ai-history/index.html

UNESCO. (n.d.). *Ethics of Artificial Intelligence*. Retrieved April 4, 2024, from UNESCO: https://www.unesco.org/en/artificial-intelligence/recommendation-ethics

University of Helsinki. (n.d.). *A Framework For AI Ethics*. Retrieved April 6, 2024, from https://ethics-of-ai.mooc.fi/chapter-1/4-a-framework-for-ai-ethics

Varadi, P. (2023, January 22). *The Rise of AI: Will Junior Developers be Left Jobless?* Retrieved April 17, 2024, from Medium: https://medium.com/@umfpeti/the-rise-of-ai-will-junior-developers-be-left-jobless-355405fa249#:~:text=One%20more%20way%20that%20AI,on%20the%20market%20may%20rise.

Velasquez, M., Andre, C., Shanks, T., S, J., & Meyer, J. (2010, January 1). What is Ethics? Retrieved from https://www.scu.edu/ethics/ethics-resources/ethical-decision-making/what-is-ethics/

Ventoniemi, J. (n.d.). *What is an AI Chatbot? Here's Everything You Need to Know*. Retrieved April 6, 2024, from https://www.giosg.com/blog/what-is-ai-chatbot#:~:text=An%20AI%20chatbot%20is%20an,involvement%20of%20a%20human%20operator.

Wooldridge, M. (2020). The Road to Conscious Machines. In M. Wooldridge, *The Road to Conscious Machines* (pp. 263-275). Pelican. Retrieved April 12, 2024

Yonatha, A., Danyllo Wagner, A., Emanuel Dantas, F., Felipe, M., Katyusco de Farias, S., Mirko, P., . . . Angelo, P. (2023). AICodeReview: Advancing Code Quality with AI-Enhanced Reviews. Retrieved April 2, 2024, from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4575968

Yue, L., Thanh, L.-C., Ratnadira, W., Chakkrit, T., Li, L., Xuan-Bach, D., & David, L. (2024). *Refining ChatGPT-Generated Code: Characterizing and Mitigating Code Quality Issues.* Retrieved April 3, 2024, from https://dl.acm.org/doi/abs/10.1145/3643674

Yutaka, W., & Mostafizer, R. (2023). *ChatGPT for Education and Research: Opportunities, Threats, and Strategies.* Retrieved March 29, 2024, from https://www.mdpi.com/2076-3417/13/9/5783

Appendix 1. Survey Questions

AI Impact on Junior Developers

Mandatory questions are marked with a star (*)

1. What is your current academic year? *

- ◯ 1
- ◯ 2
- ◯ 3
- ◯ 4
- ◯ Graduated

2. How many years of programming experience do you have? *

- ◯ Less than one
- ◯ 1
- ◯ 2
- ◯ 3
- ◯ 4
- ◯ Other

3. Have you ever used AI-generated code in your programming projects *

- ◯ Yes
- ◯ No

4. If yes, in which context have you used AI-generated code? (select all that apply) *

- [ ] Personal Projects
- [ ] Academic Projects
- [ ] Professional Projects
- [ ] Other

5. If you use AI-generated code, on a scale of 1-5, how frequently do you use AI-generated code in your programming tasks? *

- ◯ Rarely
- ◯ Occasionally
- ◯ Sometimes
- ◯ Frequently
- ◯ Always

6. How do you perceive the quality of AI-generated code compared to the code written by humans? *

- ◯ Superior
- ◯ Comparable
- ◯ Inferior

7. What benefits do you think AI-generated code offers in programming projects? *

- [ ] Time-saving
- [ ] Enhanced productivity
- [ ] Improved code quality
- [ ] No benefits
- [ ]

8. When encountering coding problems or errors, which of the following methods do you primarily rely on to seek solutions or explanations? *

- [ ] Utilizing AI chatbots or virtual assistants designed to assist with coding-related queries.
- [ ] Googling the problem or error to find the solutions or explanations from online forums, documentation, or tutorials
- [ ] Other

9. What are some challenges or limitations you encountered when using AI-generated code? *

Appendix 2. Thesis Data Management Plan

Management and Storage of Research Data

During the thesis process, research data will consist of survey responses gathered through Webropol. All data gathered will be stored securely on a password-protected device owned by the researcher. Webropol already provides encryption protection for data and storage practices. Also, access to the Webropol survey will be restricted only to those authorized or relevant to the thesis process.

Access to the survey responses and any related research data will be limited only to the researcher or any supervisor or advisor involved in the thesis. Data will be stored on personal devices and will be backed up regularly to external hard drives, no cloud storage will be used.

In case data collected by another party is used in the thesis, the terms of use of the data will be followed, and proper credit will be provided. All other data obtained from external sources will be properly cited.

Processing of Personal Data and Sensitive Data

In this thesis, personal data will be collected from participants through a survey. The personal data collected will be limited to information necessary for the study. All personal data will be handled per applicable data protection regulations, and appropriate measures will be implemented to ensure the security and confidentiality of the data. Also, consent for the participation of the survey and processing of the data will be mentioned within the survey itself.

Ownership of Thesis Data

The data and results generated from this thesis project are owned by me, the researcher (Carlos Pantin). As the main researcher and author of the thesis, I hold the intellectual property rights to the data and findings produced through the thesis process.

No other parties are involved in the thesis process. Therefore, there are no agreements or arrangements regarding ownership and use of data with other parties.

Further Use of Thesis Data After Work is Completed

The data generated from this thesis will not be utilized or made available for further use. After the thesis is completed, data collected will be securely stored for one year from the date of approval of the thesis, thus ensuring the verification of the thesis results.

After the storage period passes, all research data will be securely deleted to maintain data privacy and confidentiality. No further use or distribution of the data will take place.